

同济大学计算机系

计算机组成原理课程综合实验报告



学 号 2452769

姓 名 幸可函

专 业 计算机科学与技术

授课老师 张冬冬

一、实验内容

使用 Verilog 实现 31 条 MIPS 指令的单周期 CPU 的设计、前仿真、后仿真和下板调试运行。

二、CPU 数据通路设计

(1) 根据每条指令所涉及部件和部件的数据输入来源，画出每条指令的数据通路。

(2) 根据每条指令功能，在已形成的数据通路下，画出每条指令从取指到执行过程的指令流程图。

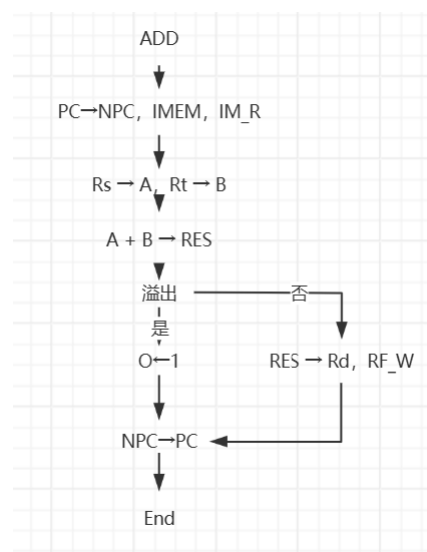
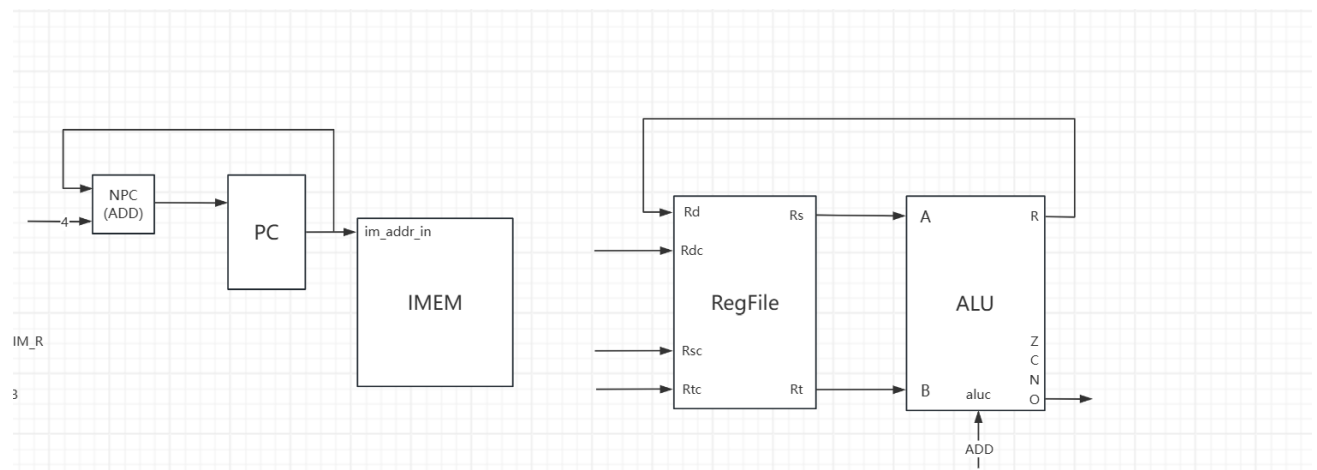
1. ADD

格式: ADD rd, rs, rt

描述: $rd \leftarrow rs + rt$

目的: 32 位数相加 将通用寄存器中存的 32 位数据 rs 与 rt 相加产生一个 32 位数据存入目标寄存器 rd。

- 如果发生了溢出，则 rd 不改变并且产生一个溢出的异常。
- 如果相加不溢出，则产生的 32 位数据直接存入目标寄存器 rd



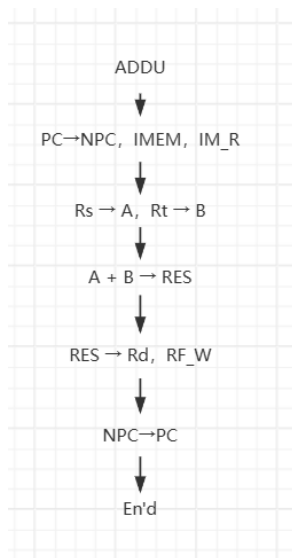
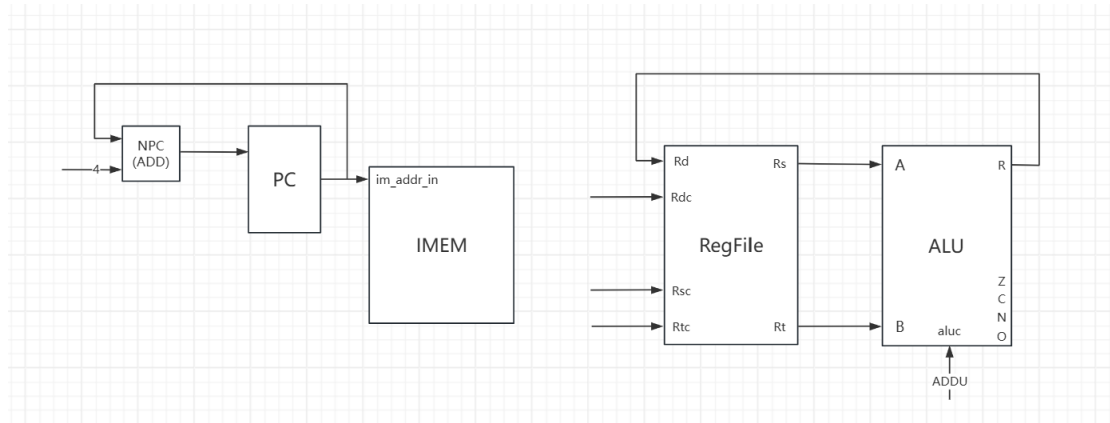
2. ADDU

格式: ADDU rd, rs, rt

目的: 32 位数据相加

描述: $rd \leftarrow rs + rt$ 将通用寄存器中存的 32 位数据 rs 与 rt 相加产生一个 32 位数据, 存入目标寄存器 rd。

在任何情况下都不会有溢出的异常



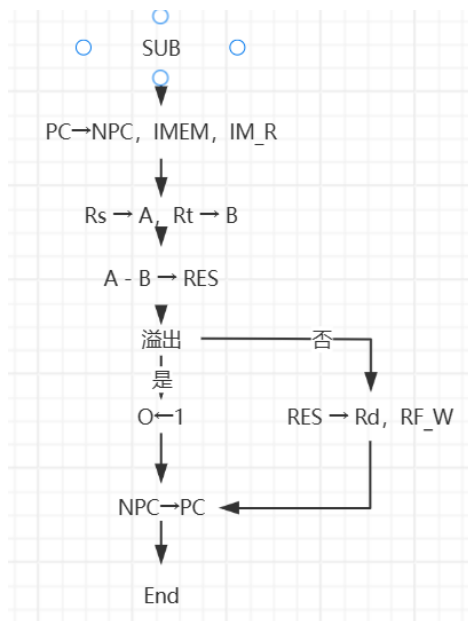
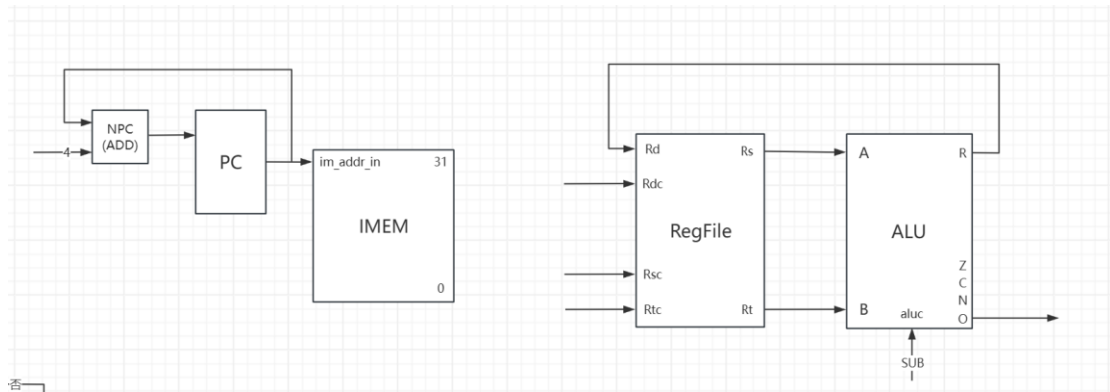
3.SUB

格式: SUB rd, rs, rt

目的: 与 32 位数相减

描述: $rd \leftarrow rs - rt$ 将通用寄存器中存的 32 位数据 rs 与 rt 相减产生一个 32 位数据存入目标寄存器 rd。

- 如果发生了溢出，则 **rd** 不改变并且产生一个溢出的异常。
- 如果不溢出，则产生的 32 位数据直接存入目标寄存器 **rd**。



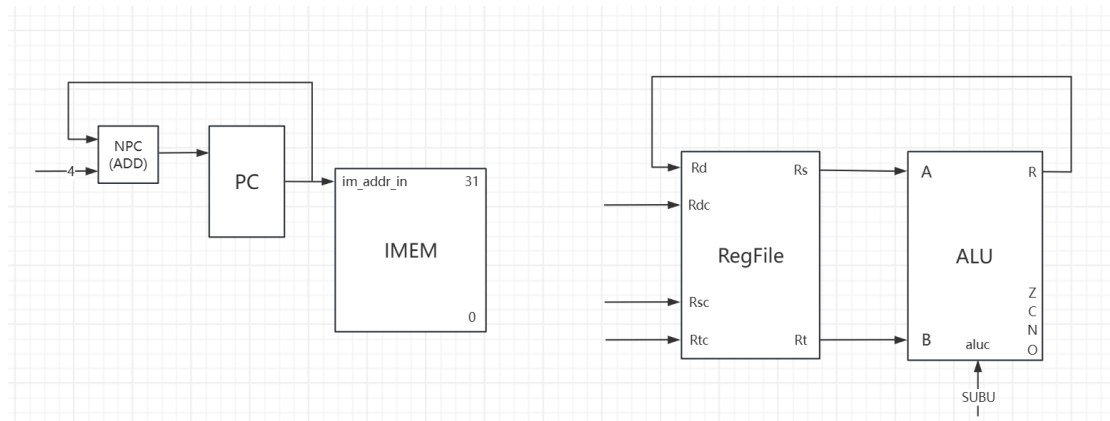
4. SUBU

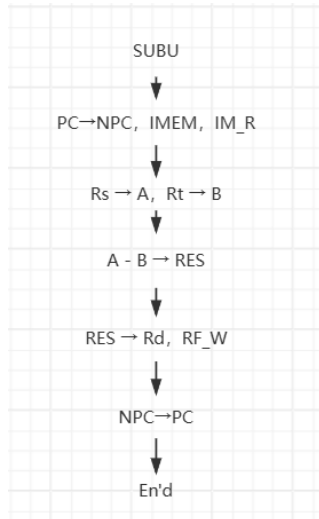
格式: SUBU rd, rs, rt

目的: 32 位数据相减

描述: $rd \leftarrow rs - rt$ 将通用寄存器中存的 32 位数据 rs 与 rt 相减产生一个 32 位数据存入目标寄存器 rd。

在任何情况下都不会有溢出的异常



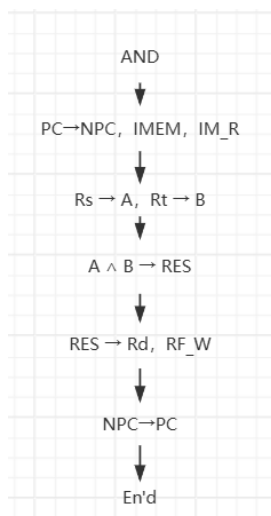
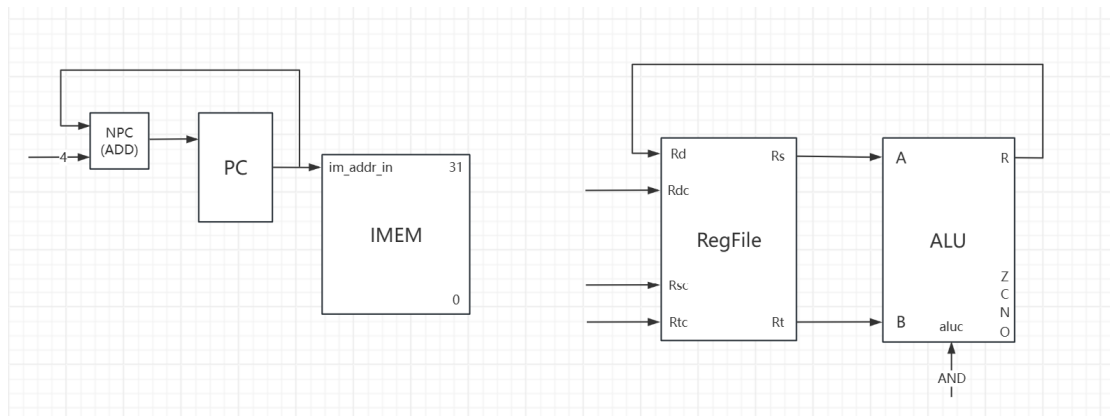


5. AND

格式: AND rd, rs, rt

目的: 按位逻辑与

描述: $rd \leftarrow rs \text{ AND } rt$ 将通用寄存器 rs 和 rd 中的数据每一位做按位与操作，将结果存入目标寄存器 rd 中。

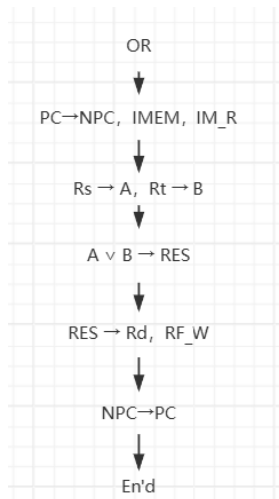
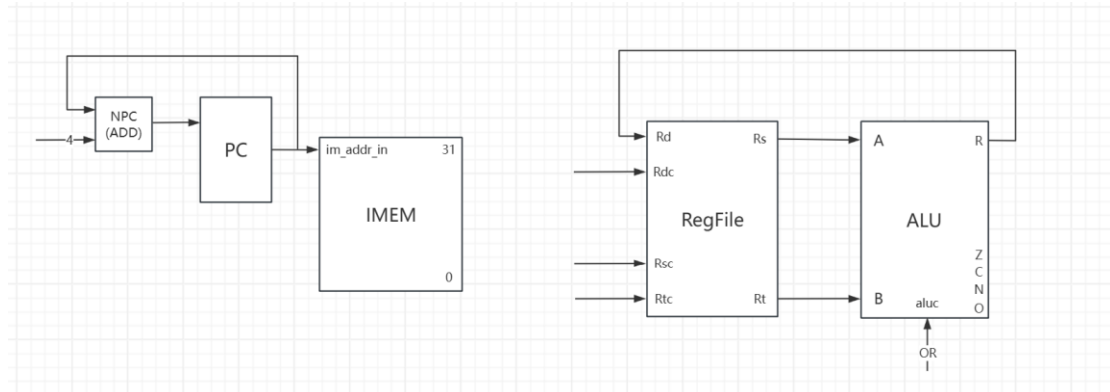


6. OR

格式: OR rd, rs, rt

目的: 按位逻辑或

描述: $rd \leftarrow rs \text{ or } rt$ 将通用寄存器 rs 和 rt 中的数据每一位做按位或操作，将结果存入目标寄存器 rd 中。



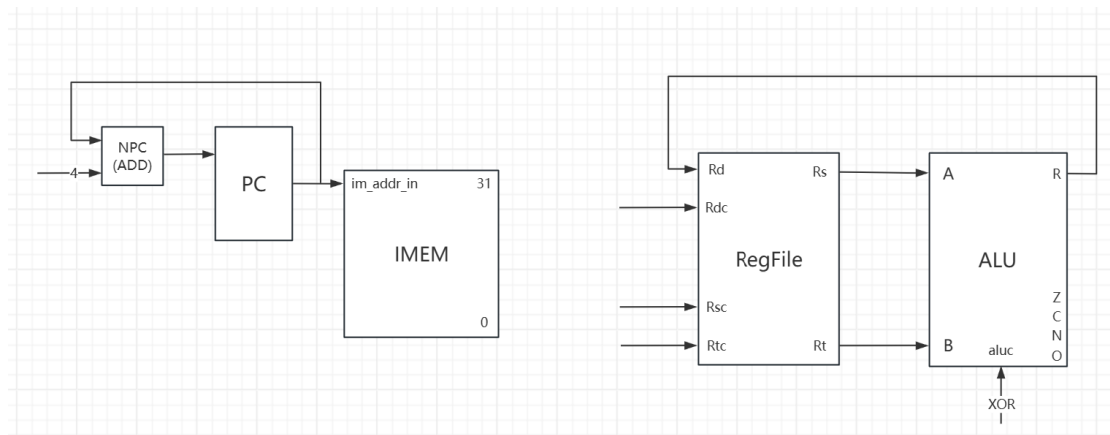
7. XOR

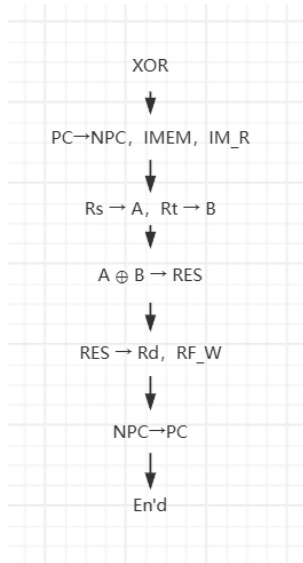
格式: XOR rd, rs, rt

目的: 按位逻辑异或

描述: $rd \leftarrow rs \text{ XOR } rt$

将通用寄存器 rs 和 rt 中的内容按位进行异或操作，将结果存入 rd 中。





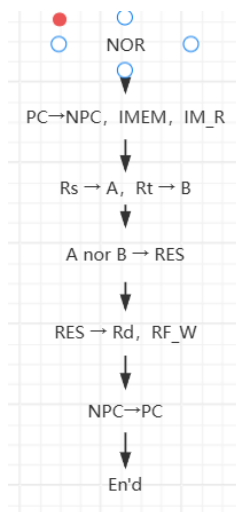
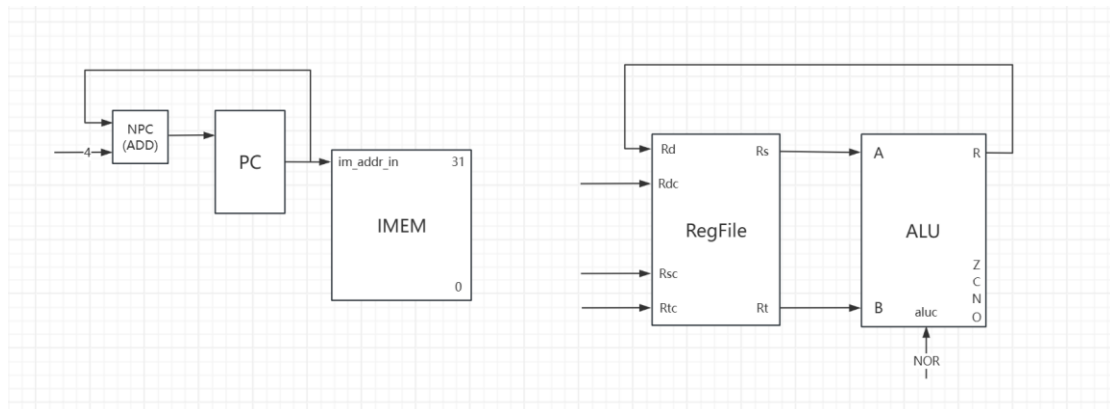
8. NOR

格式: NOR rd, rs, rt

目的: 按位逻辑或非

描述: $rd \leftarrow rs \text{ NOR } rt$

将通用寄存器 rs 和 rt 中的数据每一位做按位或非操作，将结果存入目标寄存器 rd 中。

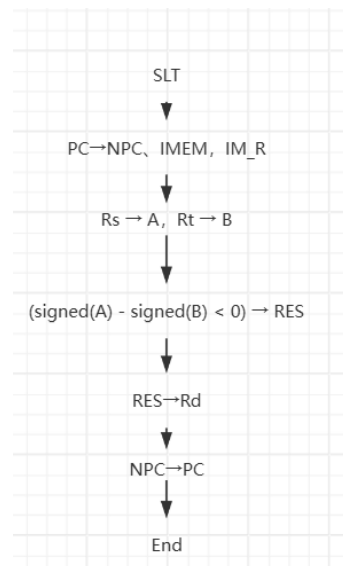
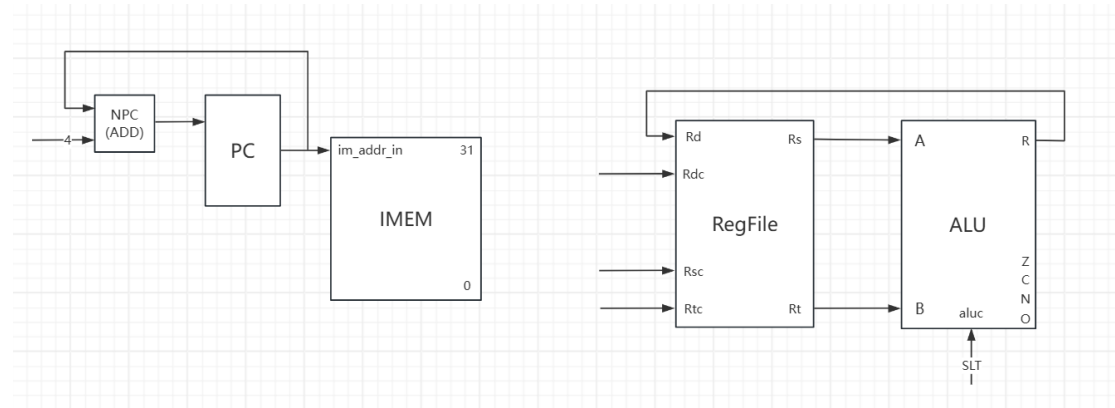


9.SLT

格式: SLT rd, rs, rt

目的: 通过小于的比较来记录结果

描述: $rd \leftarrow (rs < rt)$ 比较在 rs 和 rt 寄存器中保存的有符号数, 用 boolean 值保存结果到 rd 寄存器中。如果 rs 小于 rt, 则结果为 1, 反之结果为 0。算数比较不会引起溢出异常。

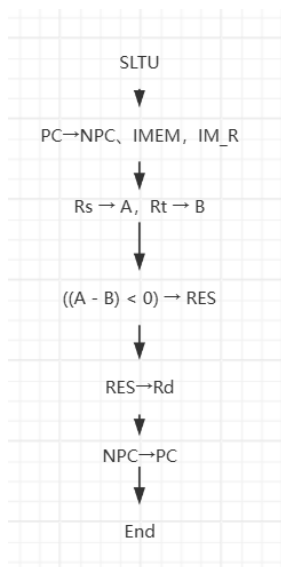
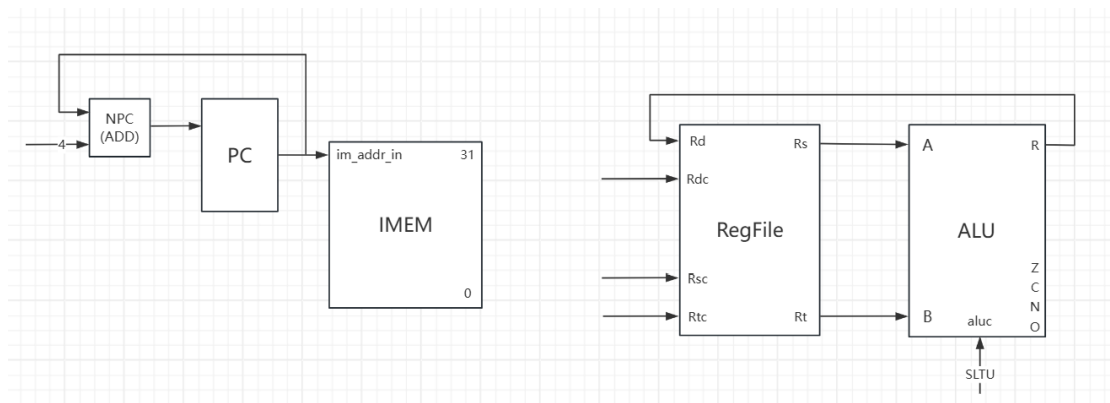


10. SLTU

格式: SLTU rd, rs, rt

目的: 通过无符号小于的比较来记录结果

描述: $rd \leftarrow (rs < rt)$ 比较在 rs 和 rt 寄存器中保存的无符号数, 用 boolean 值保存结果到 rd 寄存器中。如果 rs 小于 rt, 则结果为 1, 反之结果为 0。算数比较不会引起溢出异常。



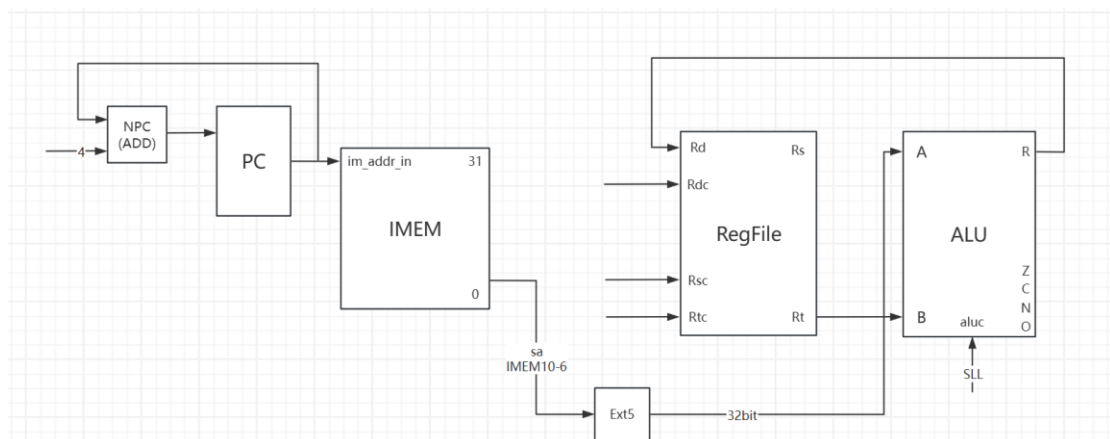
11. SLL

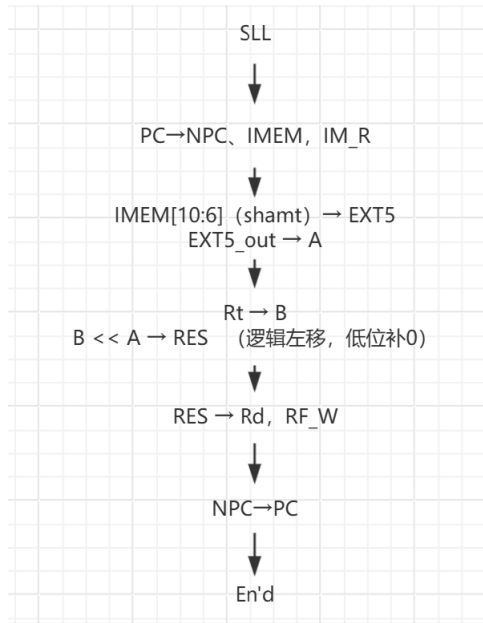
格式: SLL rd, rt, sa

目的: 通过数字填充逻辑左移

描述: $rd \leftarrow rt \ll sa$

将通用寄存器 rt 的内容左移 sa 位，空余出来的位置用 0 来填充，把结果存入 rd 寄存器。





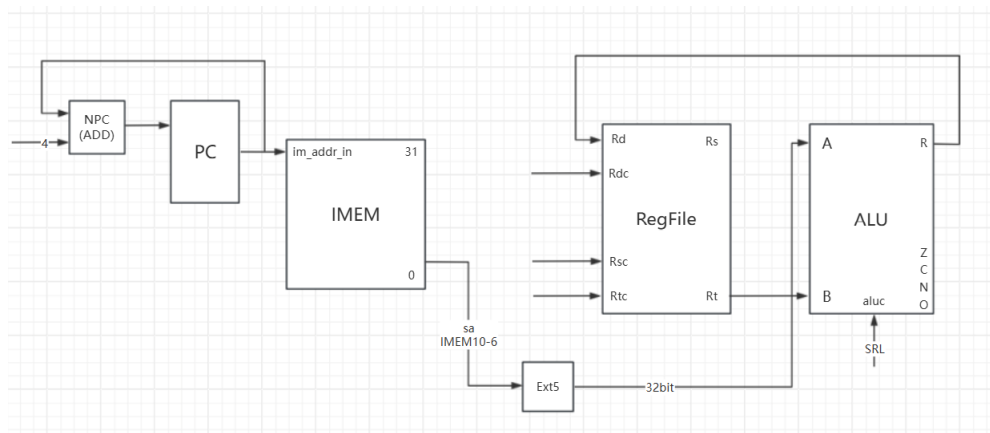
12. SRL

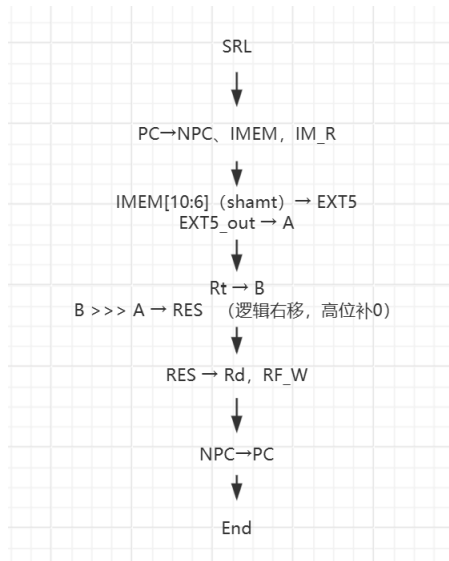
格式: SRL rd, rt, sa

目的: 通过数字填充逻辑右移

描述: $rd \leftarrow rt \gg sa \text{ (logical)}$

将通用寄存器 rt 中的 32 位内容右移 sa 位，高位用 0 来填充，结果存入通用寄存器 rd。





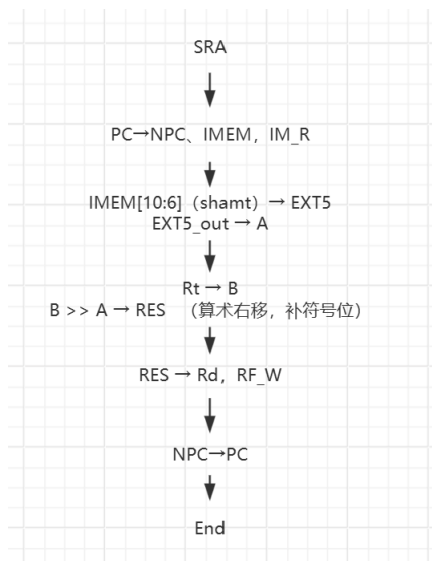
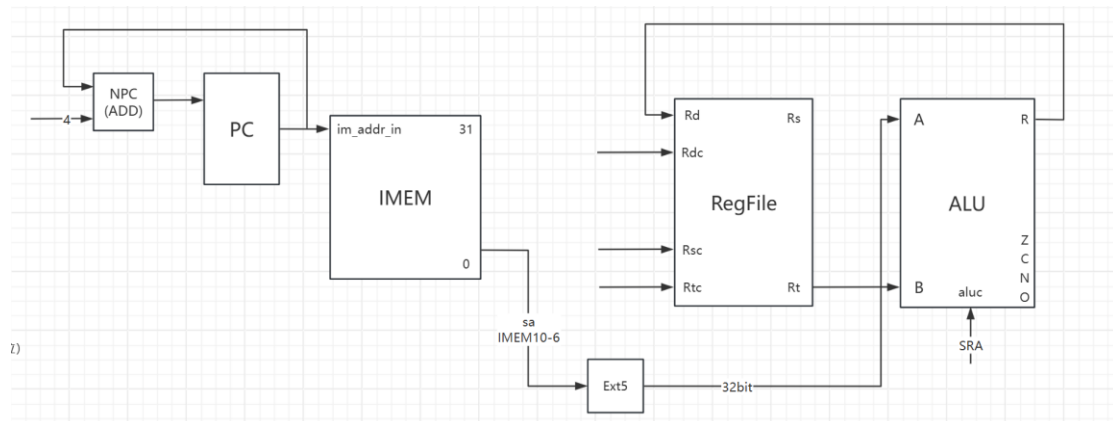
13. SRA

格式: SRA rd, rt, sa

目的: 通过数字填充算术右移

描述: $rd \leftarrow rt \gg sa \text{ (arithmetic)}$

将通用寄存器 rt 中的 32 位内容右移 sa 位，高位用 rt[31]来填充，结果存入通用寄存器 rd。



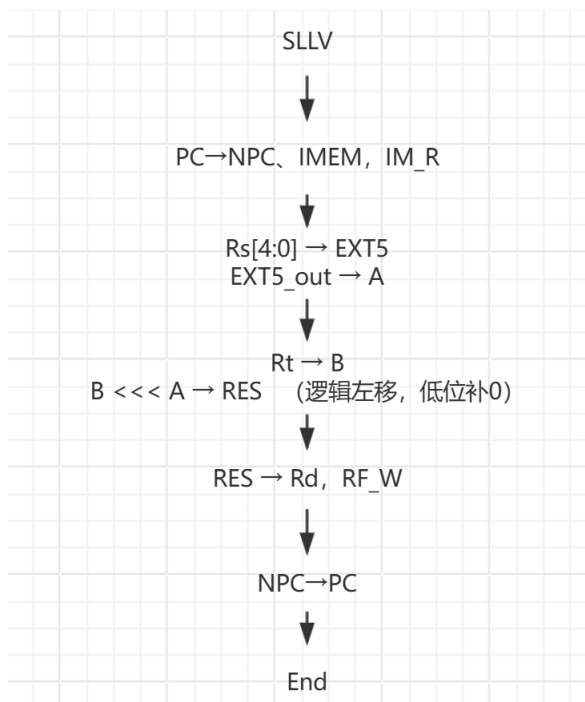
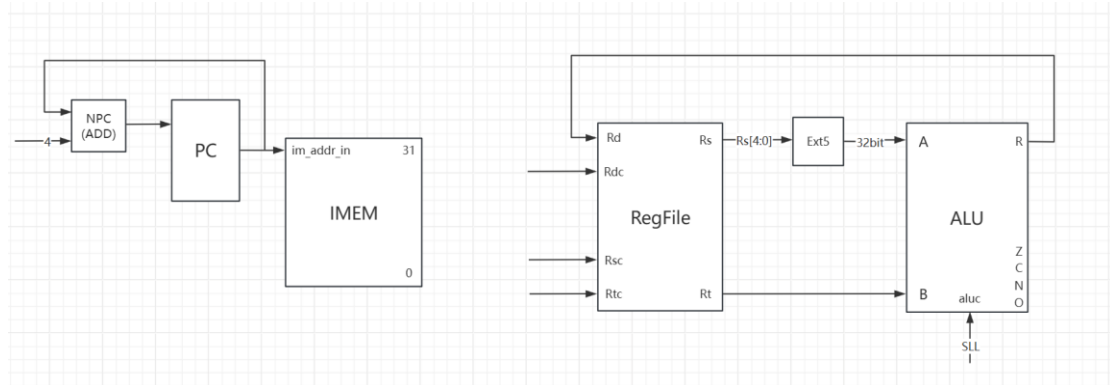
14. SLLV

格式: SLLV rd, rt, rs

目的: 通过数字填充逻辑左移

描述: $rd \leftarrow rt \ll rs$

将通用寄存器 rt 的内容逻辑左移，左移的位数保存在 rs 寄存器中，空余出来的位置用 0 来填充，把结果存入 rd 寄存器。



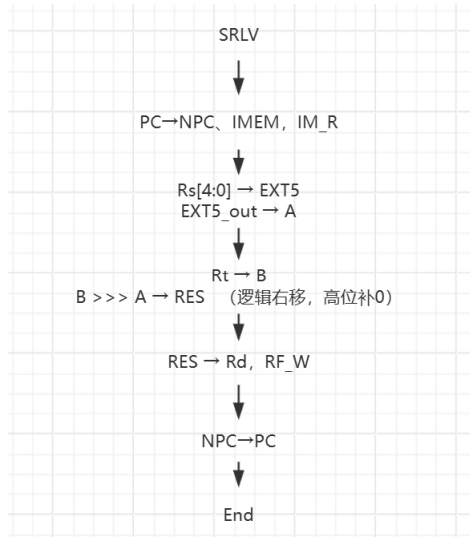
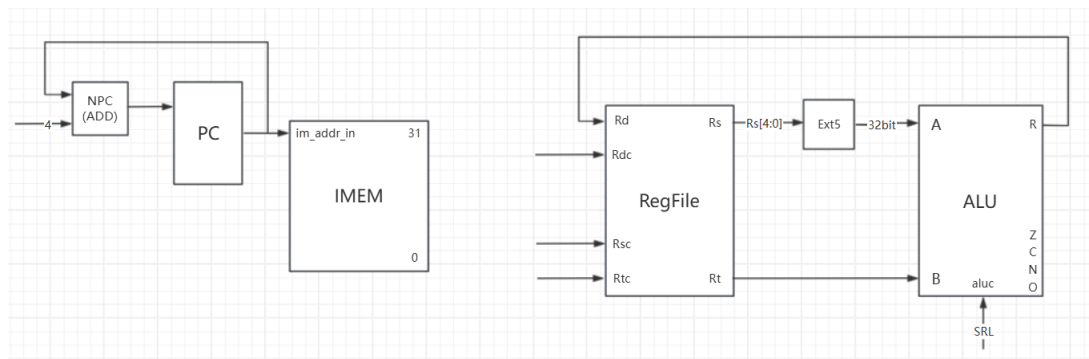
15. SRLV

格式: SRLV rd, rt, rs

目的: 通过数字填充逻辑右移

描述: $rd \leftarrow rt \gg rs \text{ (logical)}$

将通用寄存器 rt 中的 32 位内容右移，高位用 0 来填充，结果存入通用寄存器 rd。右移的位数由通用寄存器 rs 中的 0-4bit 确定。



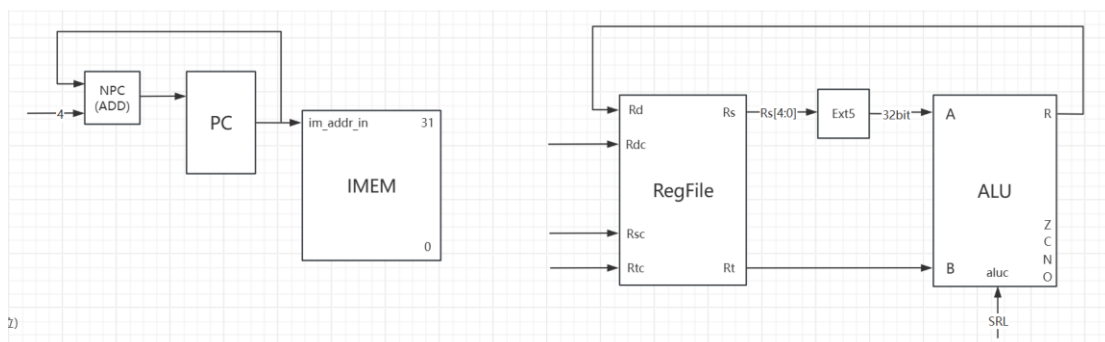
16. SRAV

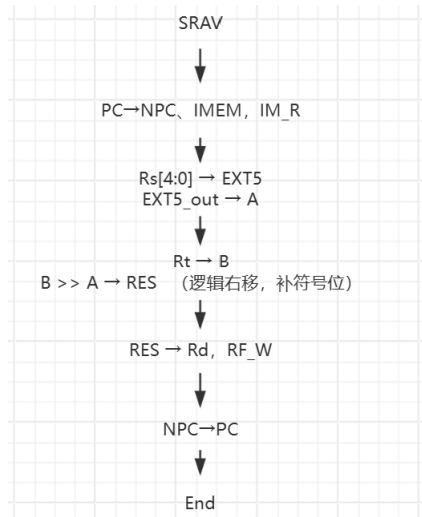
格式: SRAV rd, rt, rs

目的: 通过数字填充算术右移

描述: $rd \leftarrow rt \gg rs \text{ (arithmetic)}$

将通用寄存器 rt 中的 32 位内容右移, 高位用 rt[31]来填充, 结果存入通用寄存器 rd。右移的位数由通用寄存器 rs 中的 0-4bit 确定。





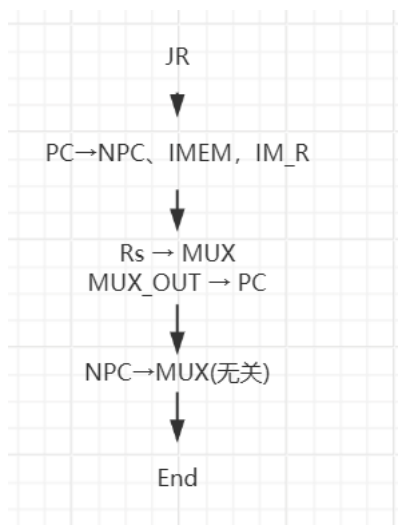
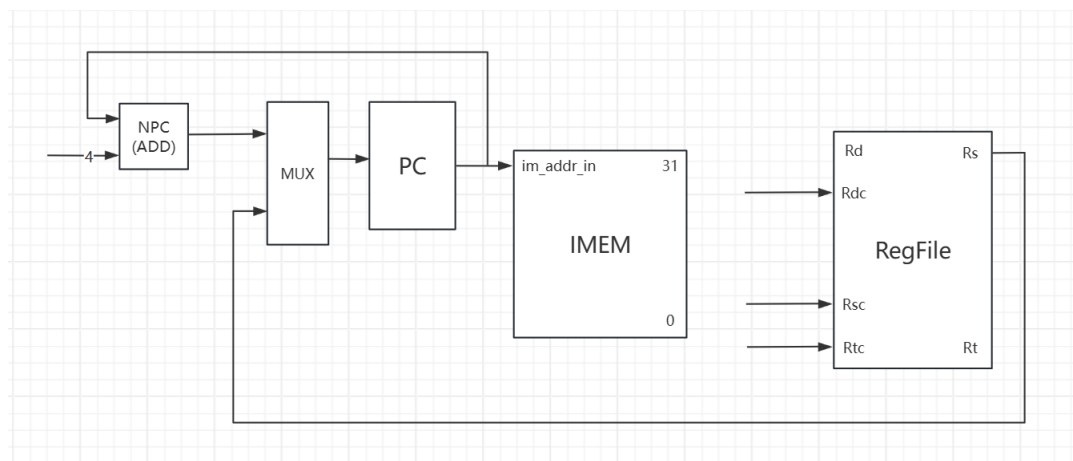
17. JR

格式: JR rs

目的: 使用寄存器的跳转指令

描述: $PC \leftarrow rs$

跳转地址存放在通用寄存器 rs 中，直接跳转到寄存器所存地址。



18. ADDI

格式: ADDI rt, rs, immediate

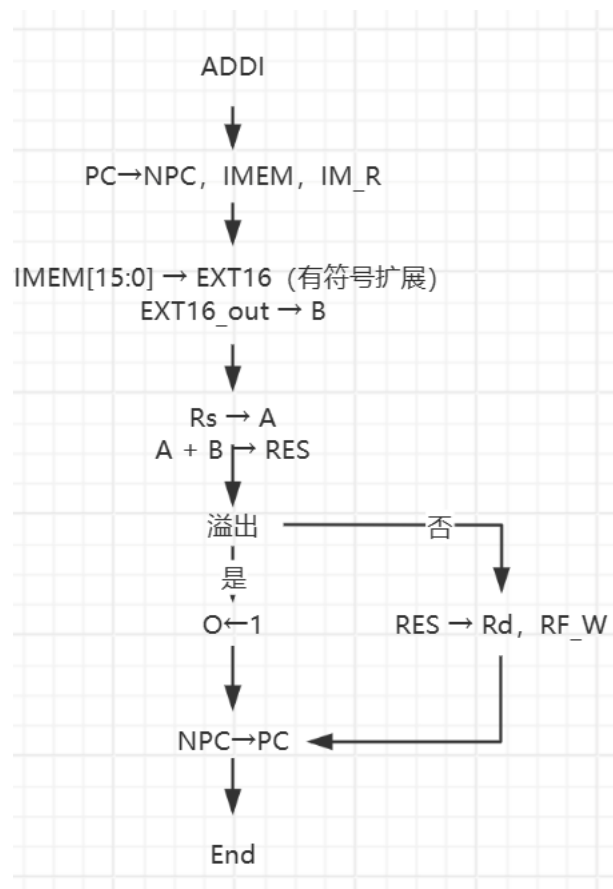
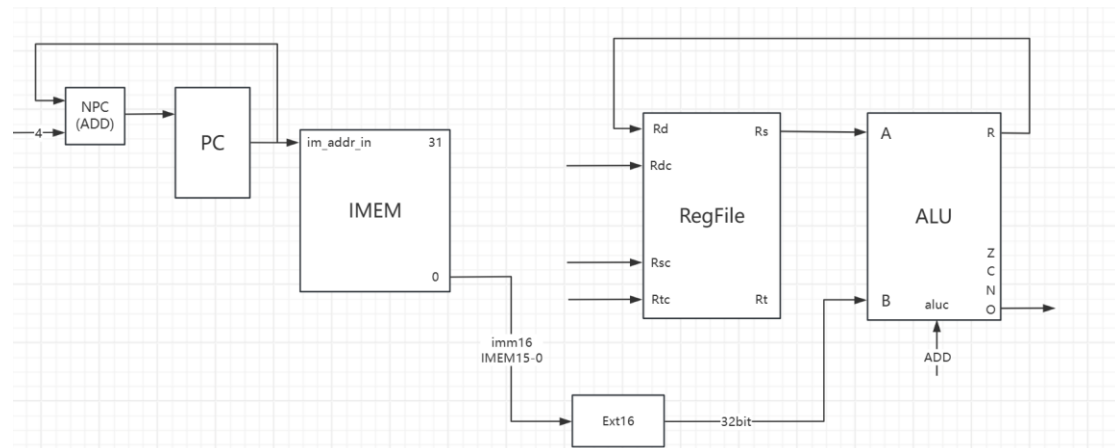
目的: 使 32 位数据与一个立即数相加

描述: $rt \leftarrow rs + \text{immediate}$

16 位有符号立即数与通用寄存器 rs 中的 32 位数相加产生一个 32 位的数存入目标寄存器 rt。

如果发生了溢出，则 rt 不改变并且产生一个溢出的异常。

如果相加不溢出，则结果存入目标寄存器 rt。



19. ADDIU

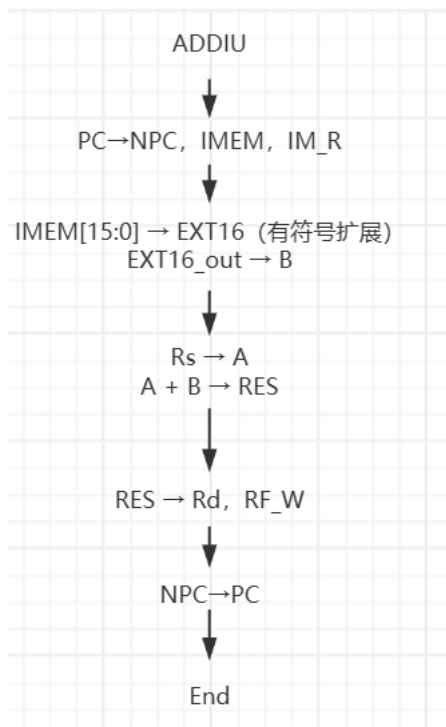
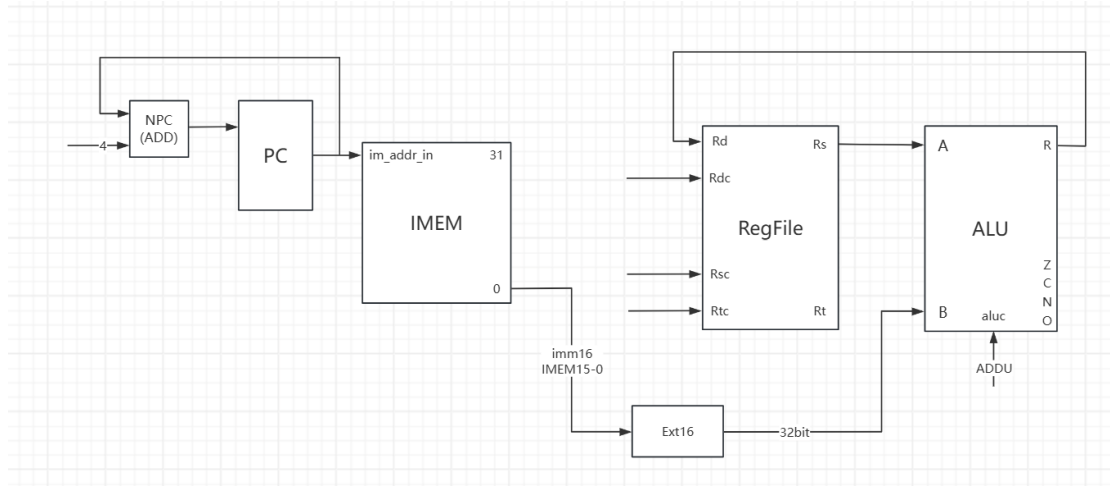
格式: ADDIU rt, rs, immediate

目的: 使 32 位数据与一个立即数相加

描述: $rt \leftarrow rs + \text{immediate}$

一个 16 位有符号的立即数与通用寄存器 rs 中的 32 位数相加产生一个 32 位的数存入目标寄存器 rt 。

在任何情况下都不会有溢出的异常。



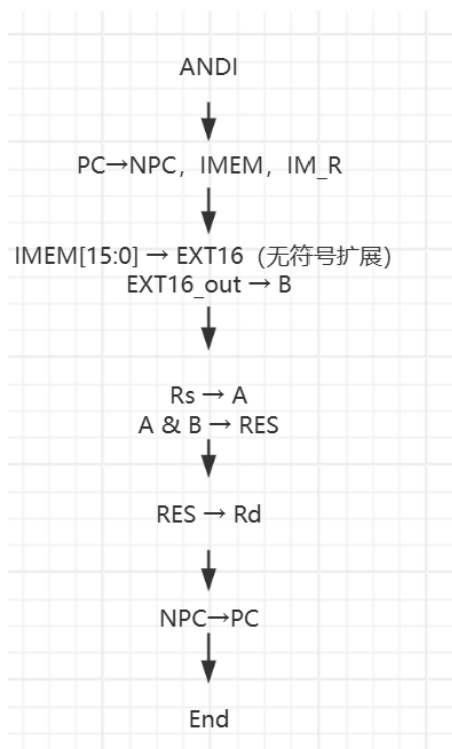
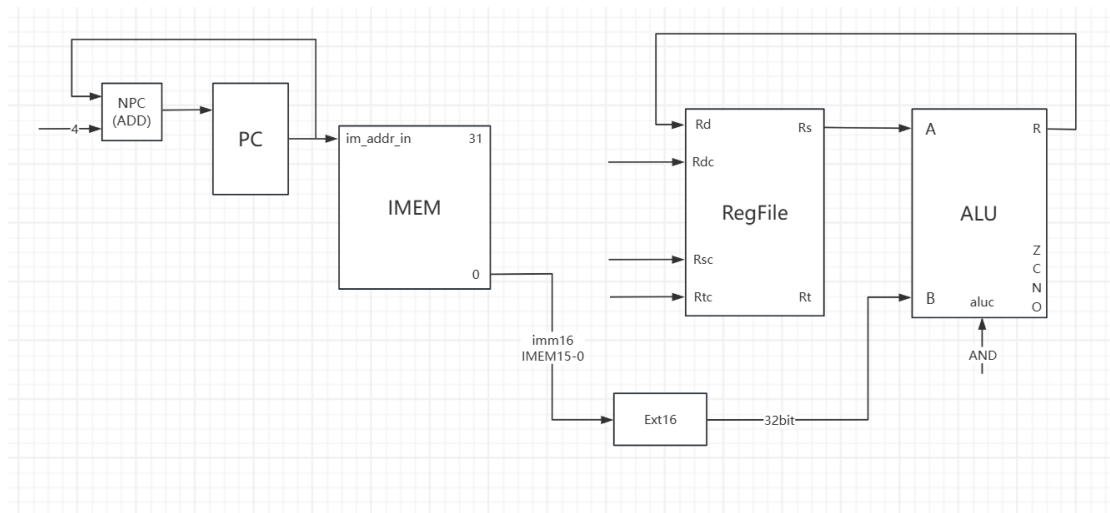
20. ANDI

格式: $\text{ANDI } rt, rs, \text{immediate}$

目的: 与一个常数做按位逻辑与

描述: $rt \leftarrow rs \text{ AND immediate}$

将 16 位立即数做 0 扩展后与通用寄存器 rs 中的 32 位数据做按位与，将结果存入目标寄存器 rt 。



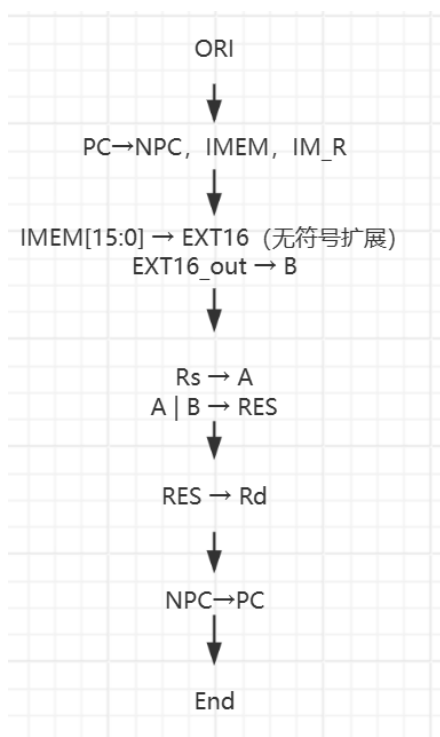
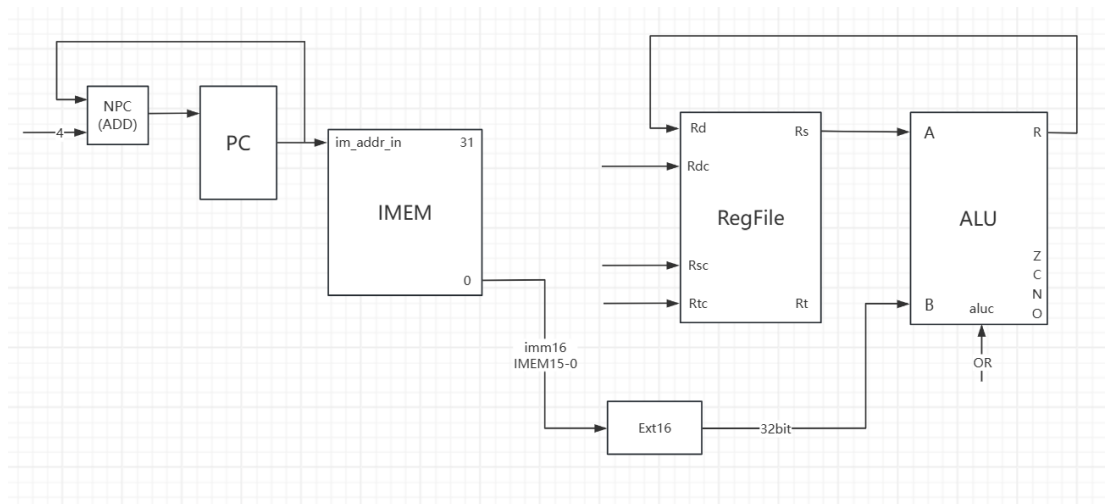
21. ORI

格式: ORI rt, rs, immediate

目的: 和一个常数做按位逻辑或

描述: $rt \leftarrow rs \text{ OR } \text{immediate}$

将通用寄存器 rs 和经过 0 扩展的立即数每一位做按位或操作，将结果存入目标寄存器 rt。



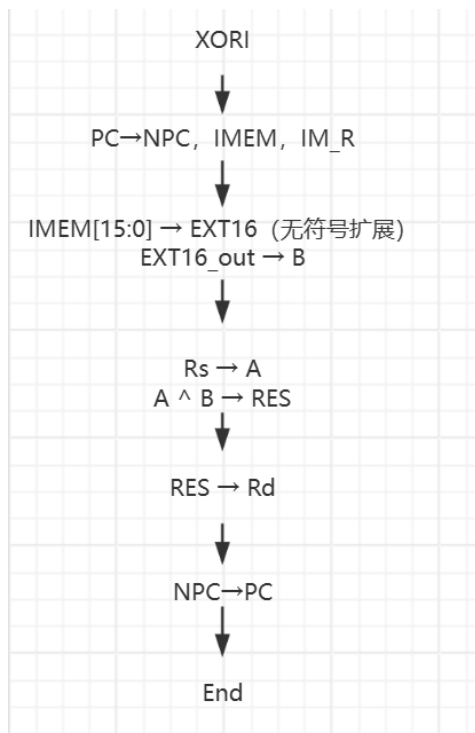
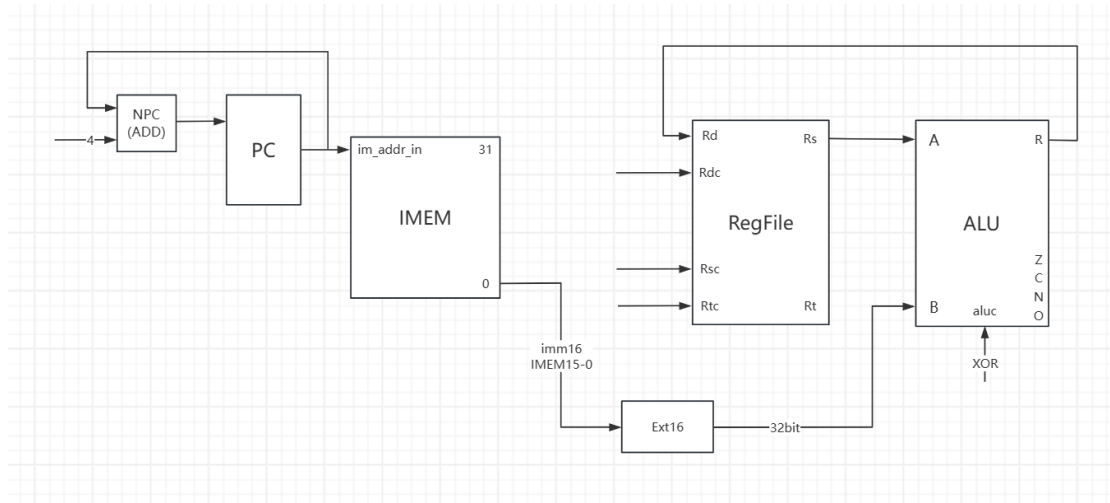
22. XORI

格式: XORI rt, rs, immediate

目的: 和一个常数做按位逻辑异或

描述: $rt \leftarrow rs \text{ XOR } \text{immediate}$

将通用寄存器 rs 和经过 0 扩展的立即数每一位做按位异或操作, 将结果存入目标寄存器 rt。



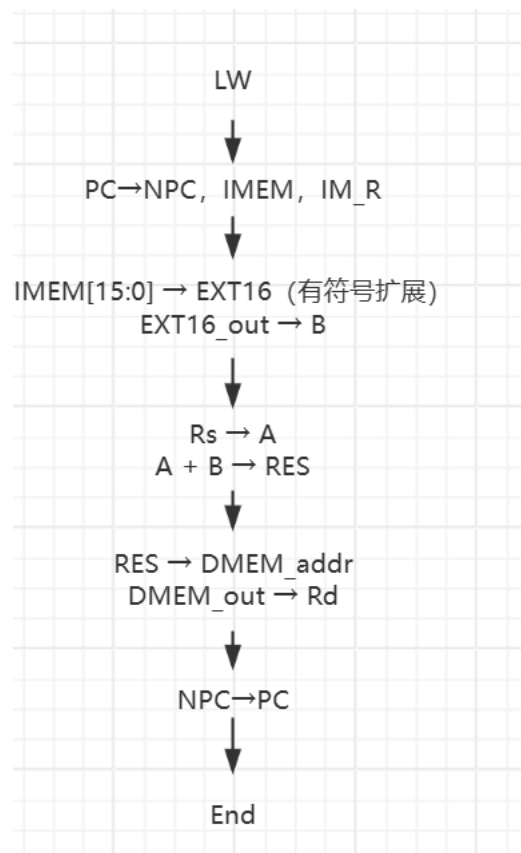
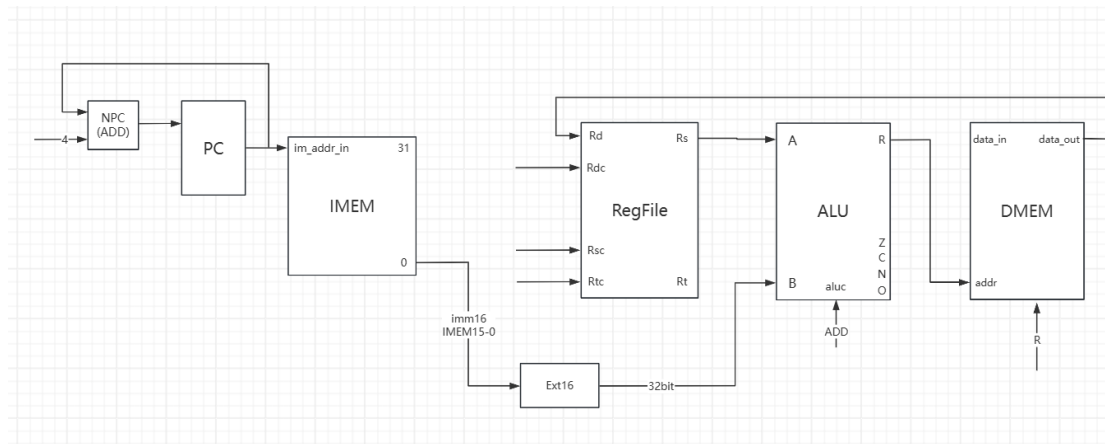
23. LW

格式: LW rt, offset(base)

目的: 从内存读取一个字的有符号数据

描述: $rt \leftarrow \text{memory}[\text{base} + \text{offset}]$

从内存中基地址加偏移量所得到的准确地址中的内容加载到通用寄存器 **rt** 中。



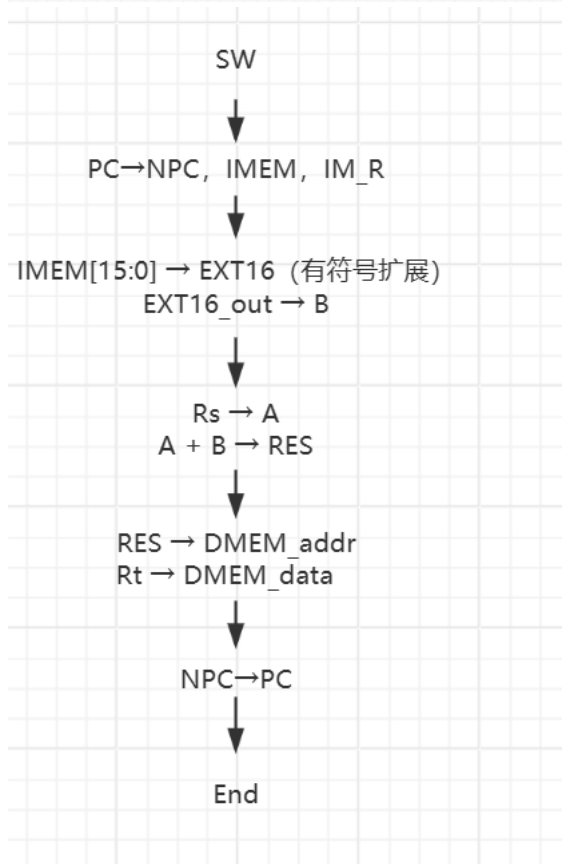
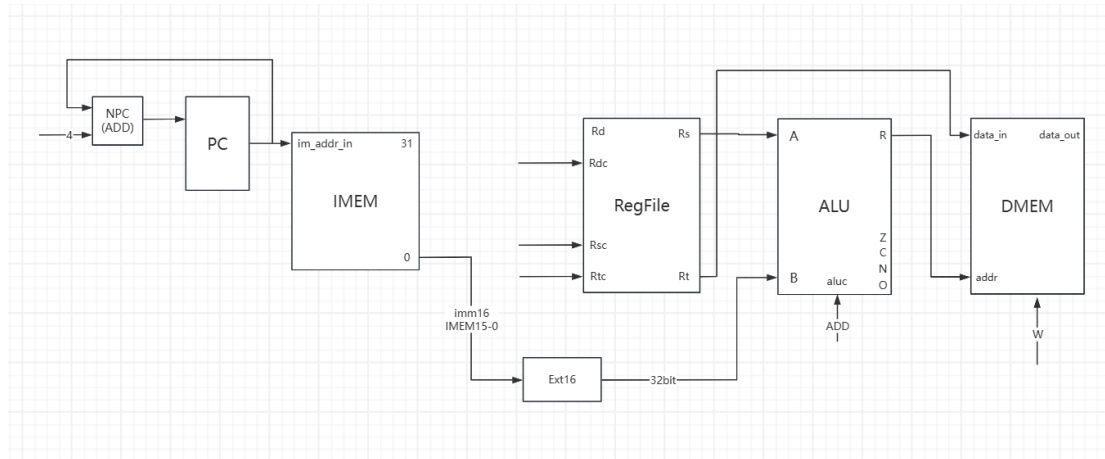
24. SW

格式: SW rt, offset(base)

目的: 存一个字到内存

描述: $\text{memory}[\text{base} + \text{offset}] \leftarrow \text{rt}$

将通用寄存器 rt 中的 32 位数据存入内存中的有效地址，有效地址由基地址和 16 位偏移量相加所得。



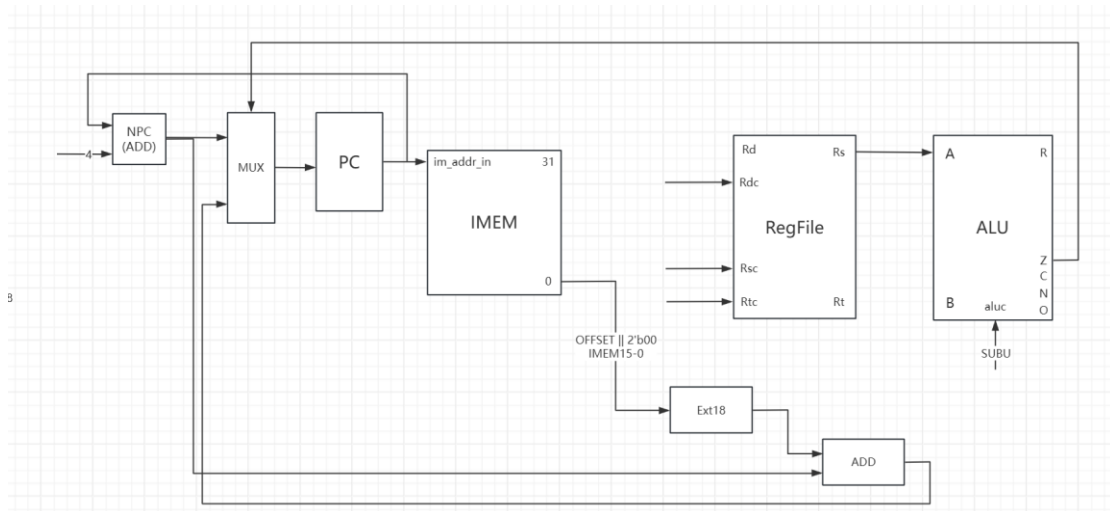
25. BEQ

格式: BEQ rs, rt, offset

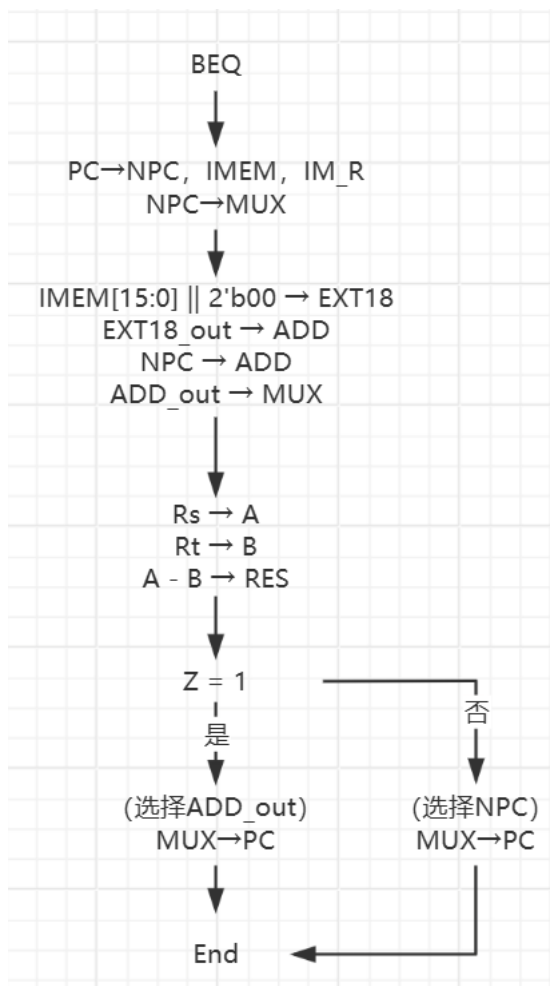
目的: 比较通用寄存器的值，然后做 pc 相关的分支跳转

描述: 如果 $rs = rt$ ，那么将 offset 左移两位，再进行符号扩展到 32 位与当前 pc 相加，形成有效转移地址，转到该地址。

如果 $rs \neq rt$ ，则继续执行下条指令。



(图中 Rt 应该连 B，这里漏掉了)



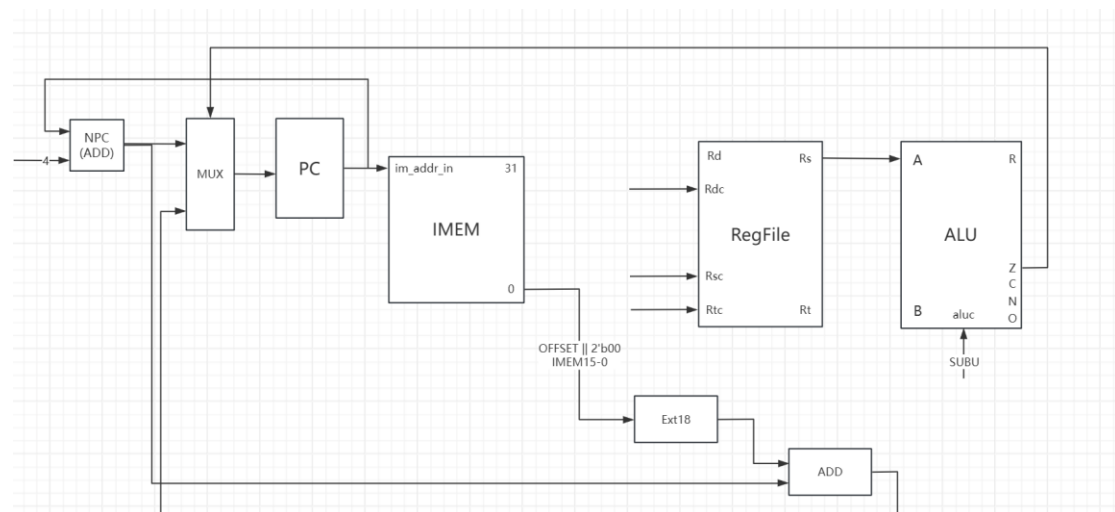
26. BNE

格式: BNE rs, rt, offset

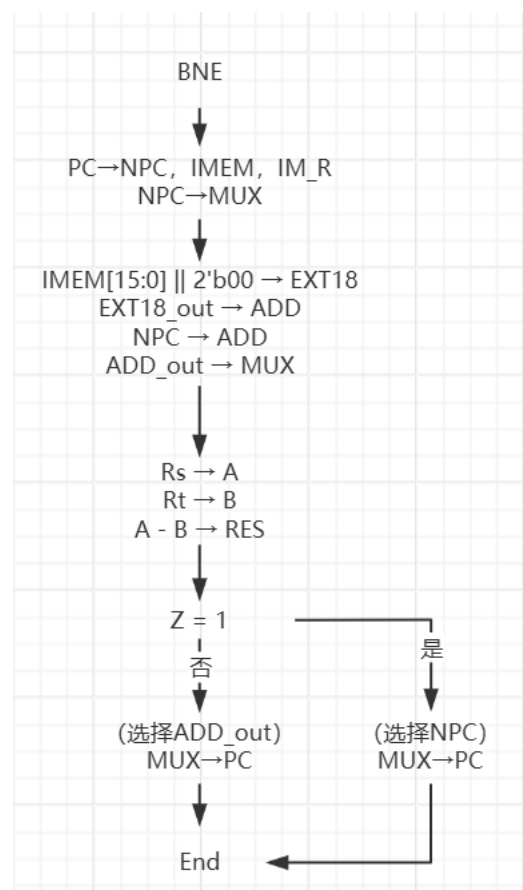
目的: 比较通用寄存器的值，然后做 pc 相关的分支跳转

描述: 如果 $rs \neq rt$ ，那么将会跳转到现在 pc 与偏移量 offset（如果是 16 位需扩展到 18 位）相加后所得的指令。

如果 $rs = rt$ ，则继续执行。



(图中 Rt 应该连 B，这里漏掉了)



27. SLTI

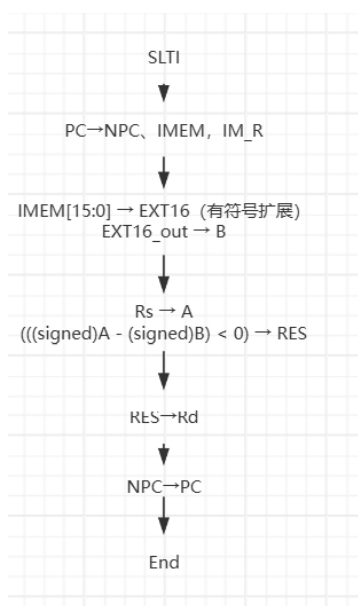
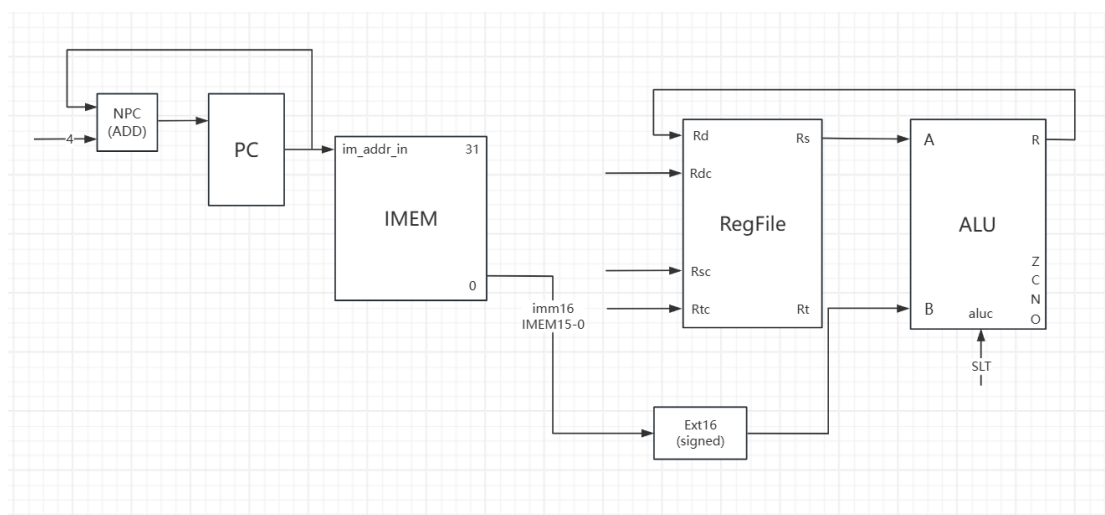
格式: SLTI rt, rs, immediate

目的: 通过跟立即数小于的比较来记录结果

描述: $rt \leftarrow (rs < \text{immediate})$

比较 rs 中的数和经过符号扩展的 16 位立即数，用 boolean 值保存结果到 rt 寄存器中。

如果 rs 中的数小于立即数，则结果为 1，反之结果为 0。算数比较不会引起溢出异常。



28. SLTIU

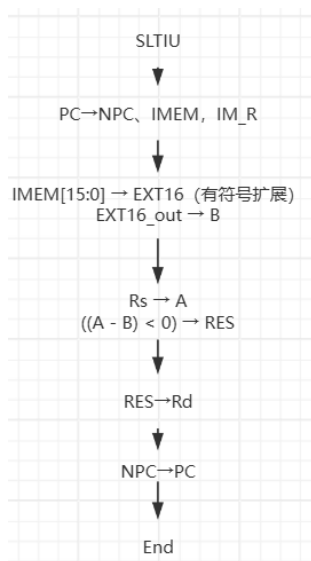
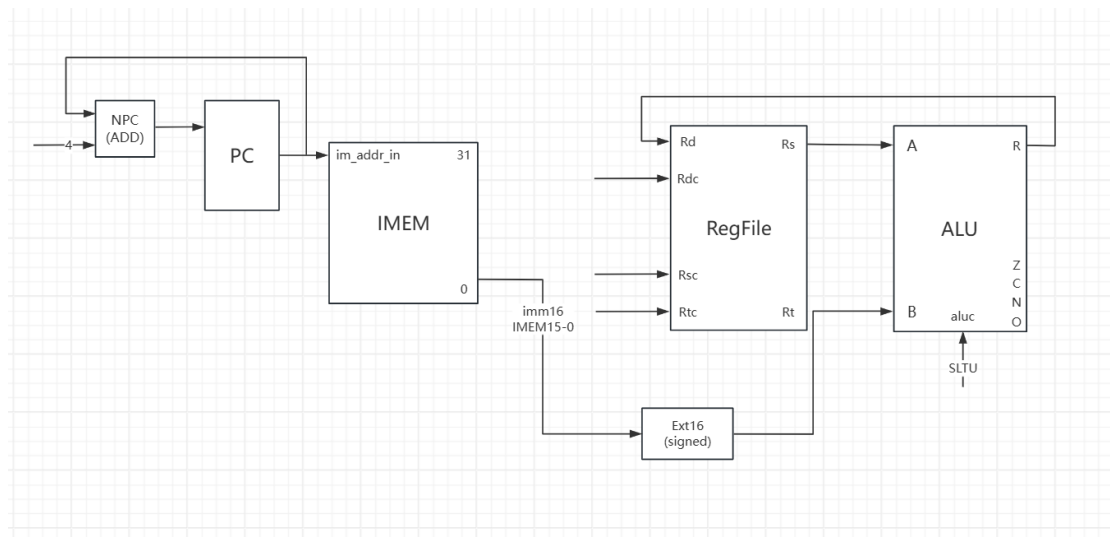
格式: SLTIU rt, rs, immediate

目的: 通过跟立即数进行无符号小于的比较来记录结果

描述: $rt \leftarrow (rs < \text{immediate})$

把 rs 中的数和经过符号扩展的 16 位立即数进行无符号小于比较，用 boolean 值保存结果到 rt 寄存器中。

如果 rs 小于符号扩展的立即数，则结果为 1，反之结果为 0。算数比较不会引起溢出异常。



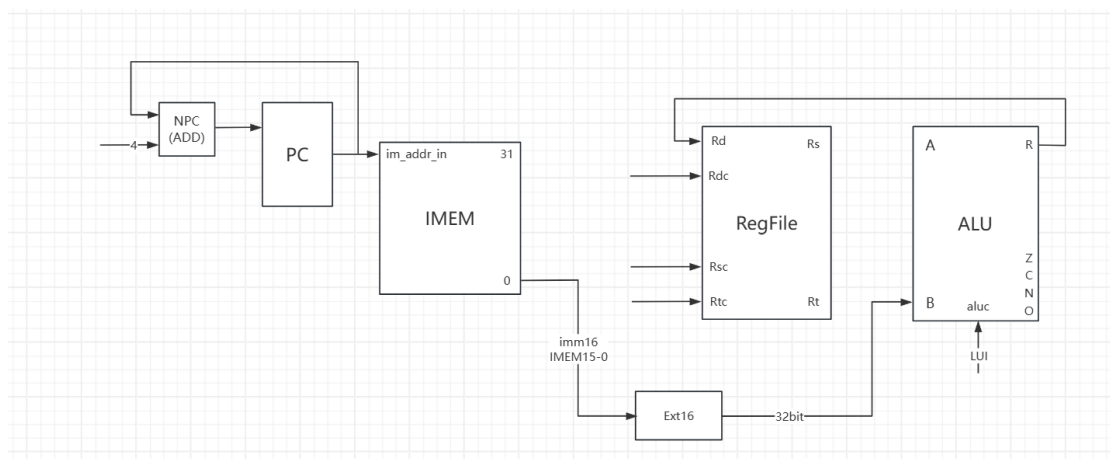
29. LUI

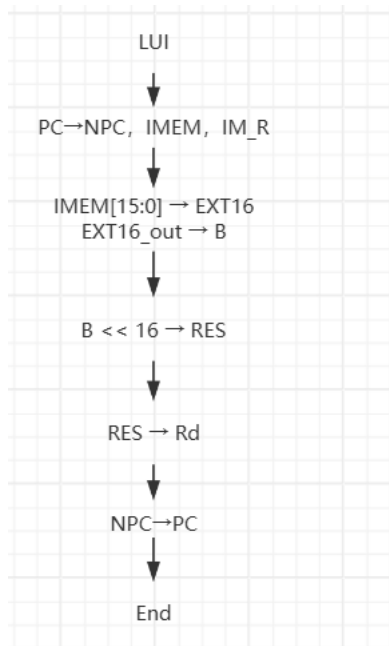
格式: LUI rt, immediate

目的: 把一个立即数载入到寄存器的高位, 低位补 0

描述: $rt \leftarrow \text{immediate} \parallel 0^{16}$

将一个 16 位的立即数载入到通用寄存器 rt 的高位, 低位补 0。



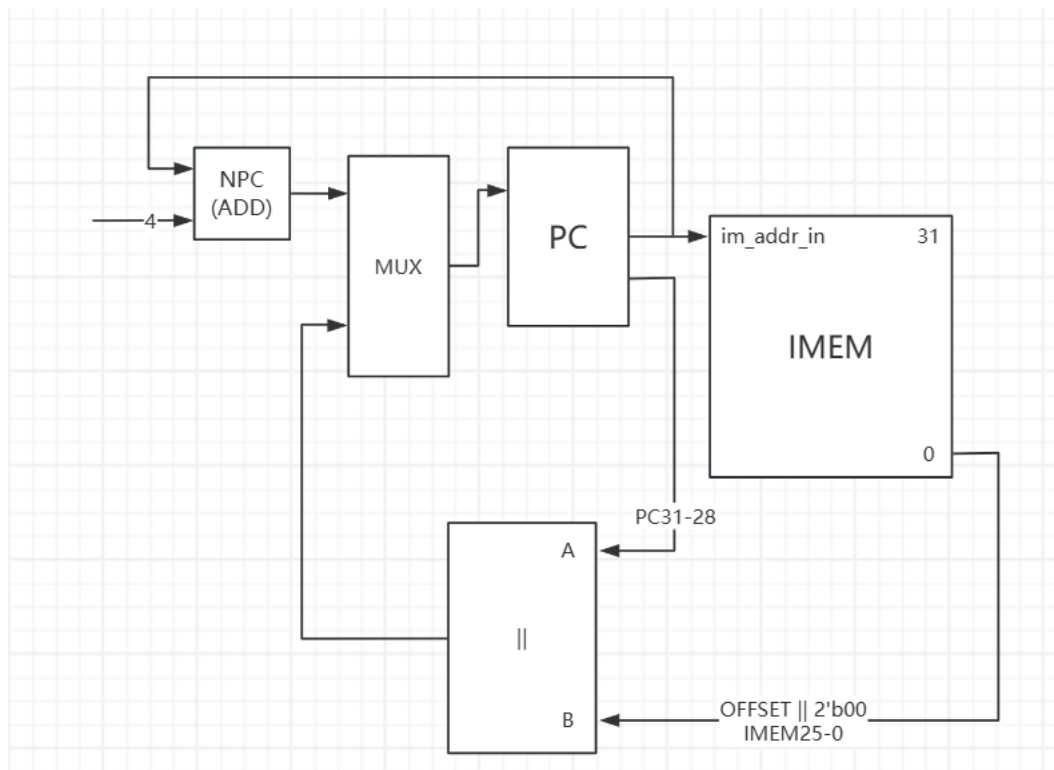


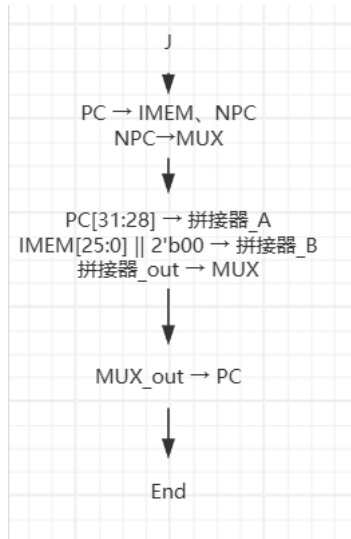
30. J

格式: J target

目的: 在 256MB 的范围内跳转

描述: 该指令无条件跳转到一个绝对地址, instr_index 有 26 位, 在左移过后访问空间能达到 256M。



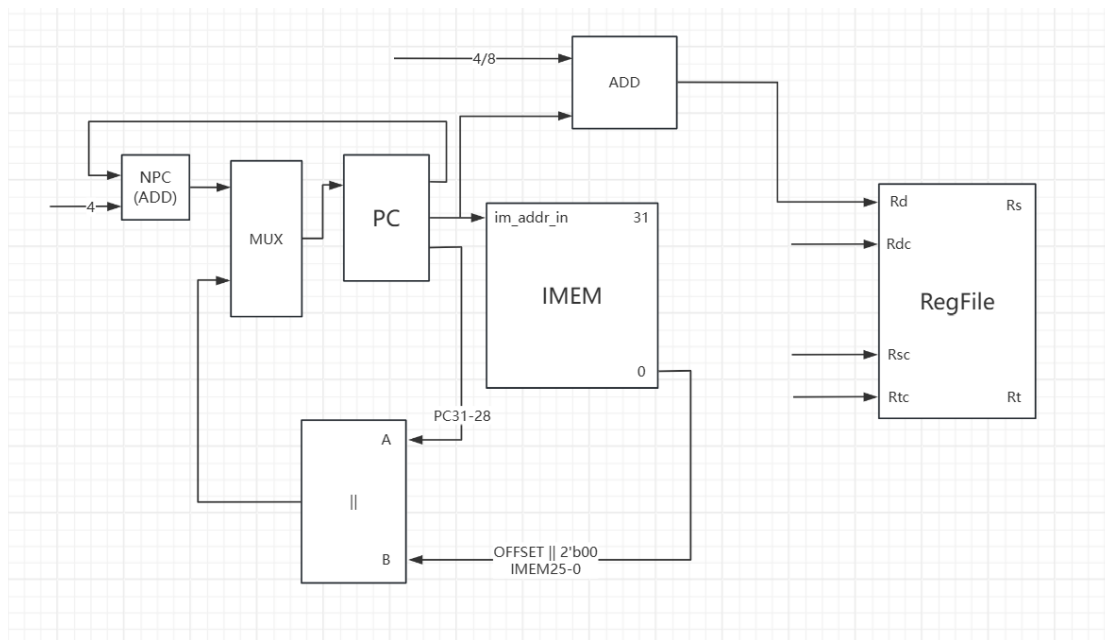


31. JAL

格式: JAL target

目的: 在 256MB 范围内执行一个过程调用

描述: 在跳转到指定地址执行子程序调用的同时, 在 31 号寄存器中存放返回地址 (当前地址后的第二条指令地址)。



(2) 根据指令操作时间表，写出每个控制信号的逻辑表达式。

假设由指令译码器翻译出的指令信号为 $i_{\dots}$ 。

选择器选择信号：

$$\begin{aligned}
 M0 &= i_{beq} \cdot alu.zero + i_{bne} \cdot \sim alu.zero \\
 M1 &= i_{sll} + i_{srl} + i_{sra} + i_{sllv} + i_{srlv} + i_{srav} \\
 M2 &= i_{sll} + i_{srl} + i_{sra} \\
 M3 &= i_{jr} \\
 M4 &= i_{addi} + i_{addiu} + i_{andi} + i_{ori} + i_{xori} \\
 &+ i_{lw} + i_{sw} + i_{slti} + i_{sltiu} + i_{lui} \\
 M5 &= i_{andi} + i_{ori} + i_{xori} + i_{lui} \\
 M6 &= i_{lw} \\
 M7 &= i_{bne} \\
 M8 &= i_j + i_{jal} \\
 M9 &= i_{jal}
 \end{aligned}$$

Regfile 写使能：

$$\begin{aligned}
 rf_w &= i_{add} + i_{addu} + i_{sub} + i_{subu} \\
 &+ i_{and} + i_{or} + i_{xor} + i_{nor} \\
 &+ i_{slt} + i_{sltu} \\
 &+ i_{sll} + i_{srl} + i_{sra} \\
 &+ i_{sllv} + i_{srlv} + i_{srav} \\
 &+ i_{addi} + i_{addiu} \\
 &+ i_{andi} + i_{ori} + i_{xori} \\
 &+ i_{lw} \\
 &+ i_{slti} + i_{sltiu} \\
 &+ i_{lui} \\
 &+ i_{jal} \\
 &= \sim(i_{jr} + i_{sw} + i_{beq} + i_{bne} + i_j)
 \end{aligned}$$

DMEM 控制信号：

$$\begin{aligned}
 dm_r &= i_{lw} \\
 dm_w &= i_{sw} \\
 dm_ena &= i_{lw} + i_{sw}
 \end{aligned}$$

ALU 控制信号：

$$\begin{aligned}
 aluc3 &= i_{lui} \\
 &+ i_{sltu} + i_{sltiu} \\
 &+ i_{slt} + i_{slti} \\
 &+ i_{sra} + i_{srav} \\
 &+ i_{srl} + i_{srlv} \\
 &+ i_{sll} + i_{sllv} \\
 \\
 aluc2 &= i_{and} + i_{andi} \\
 &+ i_{or} + i_{ori} \\
 &+ i_{xor} + i_{xori} \\
 &+ i_{nor} \\
 &+ i_{sra} + i_{srav}
 \end{aligned}$$

$$+ i_srl + i_srlv$$

$$+ i_sll + i_sllv$$

$$aluc1 = i_add + i_addi$$

$$+ i_lw + i_sw$$

$$+ i_sub$$

$$+ i_xor + i_xori$$

$$+ i_nor$$

$$+ i_sltu + i_sltiu$$

$$+ i_slt + i_slti$$

$$+ i_sll + i_sllv$$

$$aluc0 = i_subu$$

$$+ i_beq + i_bne$$

$$+ i_sub$$

$$+ i_or + i_ori$$

$$+ i_nor$$

$$+ i_slt + i_slti$$

$$+ i_srl + i_srlv$$

(3) 根据逻辑表达式，完成控制部件设计。

```
module controller(
    input i_add,
    input i_addu,
    input i_sub,
    input i_subu,
    input i_and,
    input i_or,
    input i_xor,
    input i_nor,
    input i_slt,
    input i_sltu,
    input i_sll,
    input i_srl,
    input i_sra,
    input i_sllv,
    input i_srlv,
    input i_srav,
    input i_jr,

    input i_addi,
    input i_addiu,
    input i_andi,
    input i_ori,
```

```

input i_xori,
input i_lw,
input i_sw,
input i_beq,
input i_bne,
input i_slti,
input i_sltiu,
input i_lui,

input i_j,
input i_jal,

input alu_zero,

output M0,
output M1,
output M2,
output M3,
output M4,
output M5,
output M6,
output M7,
output M8,
output M9,

output rf_w,

output dm_ena,
output dm_r,
output dm_w,

output [3:0] aluc
);

assign M0 = (i_beq & alu_zero) | (i_bne & ~alu_zero);

assign M1 = i_sll | i_srl | i_sra | i_sllv | i_srlv | i_srav;

assign M2 = i_sll | i_srl | i_sra;

assign M3 = i_jr;

assign M4 = i_addi | i_addiu | i_andi | i_ori | i_xori |
           i_lw | i_sw | i_slti | i_sltiu | i_lui;

```

```

assign M5 = i_andi | i_ori | i_xori | i_lui;

assign M6 = i_lw;

assign M7 = i_bne;

assign M8 = i_j | i_jal;

assign M9 = i_jal;

assign rf_w = i_add | i_addu | i_sub | i_subu |
              i_and | i_or | i_xor | i_nor |
              i_slt | i_sltu |
              i_sll | i_srl | i_sra |
              i_sllv | i_srlv | i_srav |
              i_addi | i_addiu |
              i_andi | i_ori | i_xori |
              i_lw |
              i_slti | i_sltiu |
              i_lui |
              i_jal;

assign dm_r = i_lw;

assign dm_w = i_sw;

assign dm_ena = i_lw | i_sw;

assign aluc[3] = i_lui |
                 i_sltu | i_sltiu |
                 i_slt | i_slti |
                 i_sra | i_srav |
                 i_srl | i_srlv |
                 i_sll | i_sllv;

assign aluc[2] = i_and | i_andi |
                 i_or | i_ori |
                 i_xor | i_xori |
                 i_nor |
                 i_sra | i_srav |
                 i_srl | i_srlv |
                 i_sll | i_sllv;

```

```
assign aluc[0] = i_subu |
                i_beq  | i_bne  |
                i_sub  |
                i_or   | i_ori  |
                i_nor  |
                i_slt  | i_slti  |
                i_srl  | i_srlv;
```

2.ADDIU

7.ANDI

/cpu_tb/inst	No Data-	32h33180000	32h3339ffff	32h3339a8000	32h3337b7fff	32h3339cf0f0	32h3339d5555	32h3339eaaaa	32h3339f1234
/cpu_tb/pc	No Data-	32h004000e0	32h004000e4	32h004000e8	32h004000ec	32h004000f0	32h004000f4	32h004000f8	32h004000fc
/cpu_tb/file_open	No Data-	32h80000003							
/cpu_tb/cnt	No Data-	32h00000038	32h00000039	32h0000003a	32h0000003b	32h0000003c	32h0000003d	32h0000003e	32h0000003f
/cpu_tb/pc_now	No Data-	32h004000dc	32h004000e0	32h004000e4	32h004000e8	32h004000ec	32h004000f0	32h004000f4	32h004000f8
/cpu_tb/inst_now	No Data-	32h32f71234	32h33180000	32h3339ffff	32h3339a8000	32h3337b7fff	32h3339cf0f0	32h3339d5555	32h3339eaaaa

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\mars_result_2_andi_mars_result.txt" --file2 "D:\Xilinx\cpu31_test\cpu_result_2_andi_cpu_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续. . .

8.LWSW

/cpu_tb/clk	1'h1								
/cpu_tb/reset	1'h0								
/cpu_tb/inst	32hac3f007c								
/cpu_tb/pc	32h00400180	32h8ff00070	32h8ffc0074	32h8ffd0078	32h8ffe0000	32h201f000f	32h3c011001	32hac3f007c	32h8c21007c
/cpu_tb/file_open	32h80000003	32h00400168	32h0040016c	32h00400170	32h00400174	32h00400178	32h0040017c	32h00400180	32h00400184
/cpu_tb/cnt	32h0000005f	32h80000003							
/cpu_tb/pc_now	32h0040017c	32h0000005a	32h0000005b	32h0000005c	32h0000005d	32h0000005e	32h0000005f	32h00000060	32h00000061
/cpu_tb/inst_now	32h3c011001	32h00400164	32h00400168	32h0040016c	32h00400170	32h00400174	32h00400178	32h0040017c	32h00400180
/cpu_tb/inst_now	32h3c011001	32h8ffa005c	32h8ff00070	32h8ffc0074	32h8ffd0078	32h8ffe0000	32h201f000f	32h3c011001	32hac3f007c

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_2_lwsu_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_2_lwsu_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续. . .

9.LWSW2

/cpu_tb/clk	1'h1								
/cpu_tb/reset	1'h0								
/cpu_tb/inst	32h20060006	32h20010001	32h20020002	32h20030003	32h20040004	32h20050005	32h20060006		
/cpu_tb/pc	32h00400018	32h00400004	32h00400008	32h0040000c	32h00400010	32h00400014	32h00400018		
/cpu_tb/file_open	32h80000003	32h80000003							
/cpu_tb/cnt	32h00000006	32h00000001	32h00000002	32h00000003	32h00000004	32h00000005	32h00000006		
/cpu_tb/pc_now	32h00400014	32h00400000	32h00400004	32h00400008	32h0040000c	32h00400010	32h00400014		
/cpu_tb/inst_now	32h20050005	32h00000000	32h20010001	32h20020002	32h20030003	32h20040004	32h20050005		

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_2_lwsu2_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_2_lwsu2_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续. . .

10.NOR

/cpu_tb/clk	1'h0								
/cpu_tb/reset	1'h0								
/cpu_tb/inst	32h03e2a827	32h03c2a027	32h03c5a027	32h03c3a027	32h03c2a827	32h03c4a827	32h03c1a827	32h03c3a827	
/cpu_tb/pc	32h0040024c	32h0040023c	32h00400240	32h00400244	32h00400248	32h0040024c	32h00400250	32h00400254	32h00400258
/cpu_tb/file_open	32h80000003	32h80000003							
/cpu_tb/cnt	32h00000093	32h0000008f	32h00000090	32h00000091	32h00000092	32h00000093	32h00000094	32h00000095	32h00000096
/cpu_tb/pc_now	32h00400248	32h00400238	32h0040023c	32h00400240	32h00400244	32h00400248	32h0040024c	32h00400250	32h00400254
/cpu_tb/inst_now	32h03c3a027	32h03c2a827	32h03c5a027	32h03c3a027	32h03c2a827	32h03c4a827	32h03c1a827	32h03c3a827	

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_2_nor_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_2_nor_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续。 . . .

11.OR

/cpu_tb/clk	1h1																			
/cpu_tb/reset	1h0																			
/cpu_tb/inst	32h03e4a825	32h03c5a025	32h03c1a025	32h03c3a025	32h03e2a825	32h03e4a825	32h03e1a825	32h03e3a825												
/cpu_tb/pc	32h00400250	32h00400240	32h00400244	32h00400248	32h0040024c	32h00400250	32h00400254	32h00400258												
/cpu_tb/file_open	32h80000003	32h80000003																		
/cpu_tb/cnt	32h00000094	32h00000090	32h00000091	32h00000092	32h00000093	32h00000094	32h00000095	32h00000096												
/cpu_tb/pc_now	32h0040024c	32h0040023c	32h00400240	32h00400244	32h00400248	32h0040024c	32h00400250	32h00400254												
/cpu_tb/inst_now	32h03e2a825	32h03c2a025	32h03c5a025	32h03c1a025	32h03c3a025	32h03e2a825	32h03e4a825	32h03e1a825												

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_2_or_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_2_or_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续。 . . .

12.ORI

	Msgs																			
/cpu_tb/clk	1h0																			
/cpu_tb/reset	1h0																			
/cpu_tb/inst	32h37deaaaa	32h375a8000	32h377b7fff	32h379cf0f0	32h37bd5555	32h37deaaaa	32h37ff1234													
/cpu_tb/pc	32h004000f8	32h004000e8	32h004000ec	32h004000f0	32h004000f4	32h004000f8	32h004000fc													
/cpu_tb/file_open	32h80000003	32h80000003																		
/cpu_tb/cnt	32h0000003e	32h0000003a	32h0000003b	32h0000003c	32h0000003d	32h0000003e	32h0000003f													
/cpu_tb/pc_now	32h004000f4	32h004000e4	32h004000e8	32h004000ec	32h004000f0	32h004000f4	32h004000f8													
/cpu_tb/inst_now	32h37bd5555	32h3739ffff	32h375a8000	32h377b7fff	32h379cf0f0	32h37bd5555	32h37deaaaa													

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_2_ori_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_2_ori_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续。 . . .

13.SLL

	Msgs																			
/cpu_tb/clk	1h1																			
/cpu_tb/reset	1h0																			
/cpu_tb/inst	32h001ff800	32h00005680	32h00005680	32h00000040	32h000000fc	32h0000c000	32h001ff840	32h001ff7c0	32h001ff800											
/cpu_tb/pc	32h00400080	32h00400064	32h00400068	32h0040006c	32h00400070	32h00400074	32h00400078	32h0040007c	32h00400080											
/cpu_tb/file_open	32h80000003	32h80000003																		
/cpu_tb/cnt	32h0000001f	32h00000019	32h0000001a	32h0000001b	32h0000001c	32h0000001d	32h0000001e	32h0000001f	32h00000020											
/cpu_tb/pc_now	32h0040007c	32h00400060	32h00400064	32h00400068	32h0040006c	32h00400070	32h00400074	32h00400078	32h0040007c											

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_2_sll_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_2_sll_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续。 . . .

14.SLLV

 /cpu_tb/dk	1'h1																			
 /cpu_tb/reset	1'h0																			
 /cpu_tb/inst	32'h003ffe804																			
 /cpu_tb/pc	32'h004000d8	32'h03e0d804	32'h0000e004	32'h201f0001	32'h03ffe804	32'h201f001f	32'h03fff004	32'h001ff804												
 /cpu_tb/file_open	32'h80000003	32'h004000cc	32'h004000d0	32'h004000d4	32'h004000d8	32'h004000dc	32'h004000e0	32'h004000e4												
 /cpu_tb/cnt	32'h00000035	32'h80000003																		
 /cpu_tb/pc_now	32'h004000d4	32'h00000033	32'h00000034	32'h00000035	32'h00000036	32'h00000037	32'h00000038	32'h00000039												
		32'h004000c8	32'h004000cc	32'h004000d0	32'h004000d4	32'h004000d8	32'h004000dc	32'h004000e0												

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_2_sllv_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_2_sllv_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致.

请按任意键继续. . .

15.SLT

 /cpu_tb/dk	1'h1																			
 /cpu_tb/reset	1'h0																			
 /cpu_tb/inst	32'h00c1202a	32'h007...	32'h009ef02a	32'h00bffa2a	32'h00c1082a	32'h00c1102a	32'h00c1182a	32'h00c1202a	32'h00c1282a											
 /cpu_tb/pc	32'h00400100	32'h004...	32'h004000ec	32'h004000f0	32'h004000f4	32'h004000f8	32'h004000fc	32'h00400100	32'h00400104											
 /cpu_tb/file_open	32'h80000003	32'h80000003																		
 /cpu_tb/cnt	32'h0000003f	32'h000...	32'h0000003b	32'h0000003c	32'h0000003d	32'h0000003e	32'h0000003f	32'h00000040	32'h00000041											
 /cpu_tb/pc_now	32'h004000fc	32'h004...	32'h004000e8	32'h004000ec	32'h004000f0	32'h004000f4	32'h004000f8	32'h004000fc	32'h00400100											
 /cpu_tb/inst_now	32'h00c1182a	32'h005...	32'h007ef82a	32'h009ef02a	32'h00bffa2a	32'h00c1082a	32'h00c1102a	32'h00c1182a	32'h00c1202a											
 /ctrl/CSR	1'h0																			

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_2_slt_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_2_slt_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致.

请按任意键继续. . .

16.SLTI

 /cpu_tb/dk	1'h1																			
 /cpu_tb/reset	1'h0																			
 /cpu_tb/inst	32'h2bbd8000	3...	32'h2b397fff	32'h2b5a8000	32'h2b7b7fff	32'h2b9c0001	32'h2bbd8000	32'h2bde7fff	32'h2bff8000											
 /cpu_tb/pc	32'h004000f4	3...	32'h004000e4	32'h004000e8	32'h004000ec	32'h004000f0	32'h004000f4	32'h004000f8	32'h004000fc											
 /cpu_tb/file_open	32'h80000003	32'h80000003																		
 /cpu_tb/cnt	32'h0000003d	3...	32'h00000039	32'h0000003a	32'h0000003b	32'h0000003c	32'h0000003d	32'h0000003e	32'h0000003f											
 /cpu_tb/pc_now	32'h004000f0	3...	32'h004000e0	32'h004000e4	32'h004000e8	32'h004000ec	32'h004000f0	32'h004000f4	32'h004000f8											
 /cpu_tb/inst_now	32'h2b9c0001	3...	32'h2b180000	32'h2b397fff	32'h2b5a8000	32'h2b7b7fff	32'h2b9c0001	32'h2bbd8000	32'h2bde7fff											
 /ctrl/CSR	1'h0																			

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_2_slti_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_2_slti_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致.

请按任意键继续. . .

17.SLTIU

 /cpu_tb/dk	1'h1																			
 /cpu_tb/reset	1'h0																			
 /cpu_tb/inst	32'h2ed68000	32...	32'h2f180000	32'h2f397fff	32'h2f5a8000	32'h2f7b7fff	32'h2f9c0001	32'h2fbd8000	32'h2fde7fff	32'h2ffff8000										
 /cpu_tb/pc	32'h004000d8	32...	32'h004000e0	32'h004000e4	32'h004000e8	32'h004000ec	32'h004000f0	32'h004000f4	32'h004000f8	32'h004000fc										
 /cpu_tb/file_open	32'h80000003	32'h80000003																		
 /cpu_tb/cnt	32'h00000036	32...	32'h00000038	32'h00000039	32'h0000003a	32'h0000003b	32'h0000003c	32'h0000003d	32'h0000003e	32'h0000003f										
 /cpu_tb/pc_now	32'h004000d4	32...	32'h004000dc	32'h004000e0	32'h004000e4	32'h004000e8	32'h004000ec	32'h004000f0	32'h004000f4	32'h004000f8										
 /cpu_tb/inst_now	32'h2eb50001	32...	32'h2ef77fff	32'h2f180000	32'h2f397fff	32'h2f5a8000	32'h2f7b7fff	32'h2f9c0001	32'h2fbd8000	32'h2fde7fff										

执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_2_sltiu_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_2_sltiu_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致.

请按任意键继续. . .

18.SLTU

- cpu_tb/dk - cpu_tb/reset - cpu_tb/first - cpu_tb/jc - cpu_tb/file_open - cpu_tb/cnt - cpu_tb/jc_now - cpu_tb/first_now	1h1 1h0 32h009ef02b 32h004000ec 32h80000003 32h0000003b 32h004000e8 32h007de82b	32h009ef02b 32h004000ec 32h80000003 32h0000003b 32h004000e8 32h007de82b	32h00bfff82b 32h004000f0 32h0000003c 32h0000003c 32h004000ec 32h009ef02b	32h00c1082b 32h004000f4 32h0000003d 32h0000003d 32h004000f4 32h00bfff82b	32h00c1182b 32h004000f8 32h0000003e 32h0000003f 32h004000f8 32h00c1082b	32h00c1202b 32h00400100 32h00000040 32h00000040 32h004000fc 32h00c1182b	32h00c1282b 32h00400104 32h00000041 32h00000041 32h00400100 32h00c1202b
---	--	--	---	---	--	--	--

```
执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result\2_sltu_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result\2_sltu_mars_result.txt" --display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续.

19.SRA

[illegible]

```
执行命令: txt_compare -file1 "D:\Xilinx\cpu31_test\cpu_result\2_sra_cpu_result.txt" -file2 "D:\Xilinx\cpu31_test\mars_result\2_sra_mars_result.txt" --display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续. . .

20.SRAV

[illegible]

```
执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result\2_srav_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mar
s_result\2_srav_mars_result.txt" --display detailed
```

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续

21.SRL

[illegible]

```
执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result\_2_srl_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result\_2_srl_mars_result.txt" --display detailed

比较结果输出:

=====
在指定检查条件下完全一致.
=====

请按任意键继续. . .
```

22.SRLV

 /cpu_tb/dk

 /cpu_tb/reset

 /cpu_tb/inst

 /cpu_tb/pc

 /cpu_tb/file_open

 /cpu_tb/inst

 /cpu_tb/pc_now

 /cpu_tb/inst_now

 /cpu_tb/inst

1h1

1h0

32h201f0001

32h004000d4

32h80000003

32h00000035

32h004000d0

32h0000e006

1h0

32h03e04806	32h0000e006	32h201f0001	32h03ffe806	32h201f001f	32h03fff006	32h001ff806			
32h004000cc	32h004000d0	32h004000d4	32h004000d8	32h004000dc	32h004000e0	32h004000e4			
32h80000003									
32h00000033	32h00000034	32h00000035	32h00000036	32h00000037	32h00000038	32h00000039			
32h004000c8	32h004000cc	32h004000d0	32h004000d4	32h004000d8	32h004000dc	32h004000e0			
32h201f001f	32h03e04806	32h0000e006	32h201f0001	32h03ffe806	32h201f001f	32h03fff006			

```
执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result\_2_srlv_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result\_2_srlv_mars_result.txt" --display detailed

比较结果输出:

=====
在指定检查条件下完全一致.
=====

请按任意键继续. . .
```

23.SUB

 /cpu_tb/dk

 /cpu_tb/reset

 /cpu_tb/inst

 /cpu_tb/pc

 /cpu_tb/file_open

 /cpu_tb/inst

 /cpu_tb/pc_now

 /cpu_tb/inst_now

 /cpu_tb/inst

1h1

1h0

32h03c1f022

32h004001c8

32h80000003

32h00000072

32h004001c4

32h03c0f022

cpu

32h03e0e822	32h03a1e822	32h03a2e822	32h03c0f022	32h03c1f022	32h03c2f022	32h03e0f822	32h03e1f822	32h03e2f822	
32h004001b8	32h004001bc	32h004001c0	32h004001c4	32h004001c8	32h004001cc	32h004001d0	32h004001d4	32h004001d8	
32h80000003									
32h0000005e	32h0000006f	32h00000070	32h00000071	32h00000072	32h00000073	32h00000074	32h00000075	32h00000076	
32h004001b4	32h004001b8	32h004001bc	32h004001c0	32h004001c4	32h004001c8	32h004001cc	32h004001d0	32h004001d4	
32h0382e022	32h03a1e822	32h03a2e822	32h03a3e822	32h03c0f022	32h03c1f022	32h03c2f022	32h03e0f822	32h03e1f822	

```
执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\mars_result\_2_sub_mars_result.txt" --file2 "D:\Xilinx\cpu31_test\cpu_result\_2_sub_cpu_result.txt" --display detailed

比较结果输出:

=====
在指定检查条件下完全一致.
=====

请按任意键继续. . .
```

24.SUBU

 /cpu_tb/dk

 /cpu_tb/reset

 /cpu_tb/inst

 /cpu_tb/pc

 /cpu_tb/file_open

 /cpu_tb/inst

 /cpu_tb/pc_now

 /cpu_tb/inst_now

 /cpu_tb/inst

1h1

1h0

32h03e0f823

32h00400244

32h80000003

32h00000091

32h00400240

32h03c3f023

1h0

32h03c...	32h03c2f023	32h03c3f023	32h03e0f823	32h03e1f823	32h03e2f823	32h03e3f823			
32h004...	32h0040023c	32h00400240	32h00400244	32h00400248	32h0040024c	32h00400250			
32h80000003									
32h000...	32h0000008f	32h00000090	32h00000091	32h00000092	32h00000093	32h00000094			
32h004...	32h00400238	32h0040023c	32h00400240	32h00400244	32h00400248	32h0040024c			
32h03c...	32h03c1f023	32h03c2f023	32h03c3f023	32h03e0f823	32h03e1f823	32h03e2f823			

```
执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result\_2_subu_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result\_2_subu_mars_result.txt" --display detailed

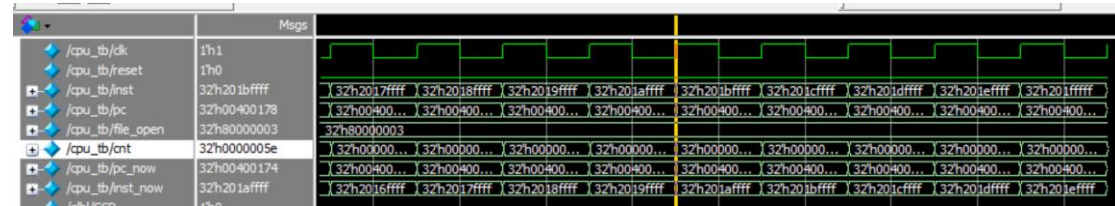
比较结果输出:

=====
在指定检查条件下完全一致.
=====

请按任意键继续. . .
```

25.XOR

29.J



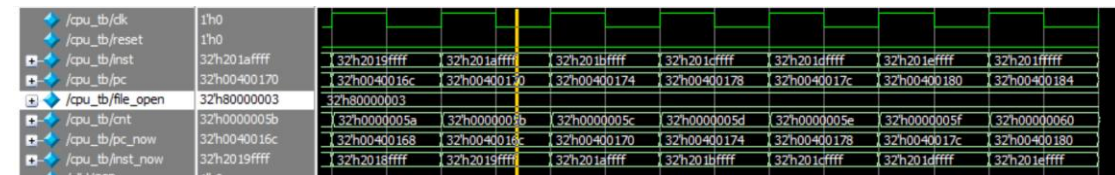
执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_3_j_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_3_j_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续. . .

30.JAL



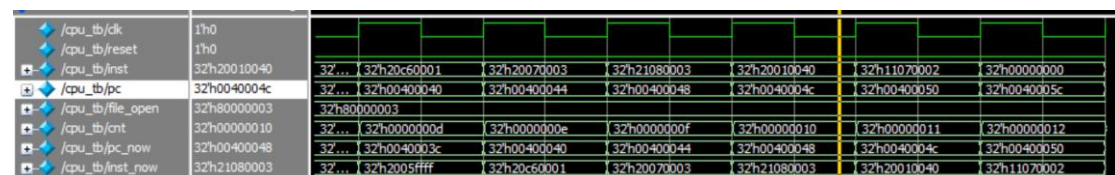
执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\cpu_result_3_jal_cpu_result.txt" --file2 "D:\Xilinx\cpu31_test\mars_result_3_jal_mars_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续. . .

31.JR



执行命令: txt_compare --file1 "D:\Xilinx\cpu31_test\mars_result_4_jr_mars_result.txt" --file2 "D:\Xilinx\cpu31_test\cpu_result_4_jr_cpu_result.txt" --display detailed

比较结果输出:

在指定检查条件下完全一致。

请按任意键继续. . .

*前仿真所用 mips_31_mars_simulate.coe 对应的部分波形图:



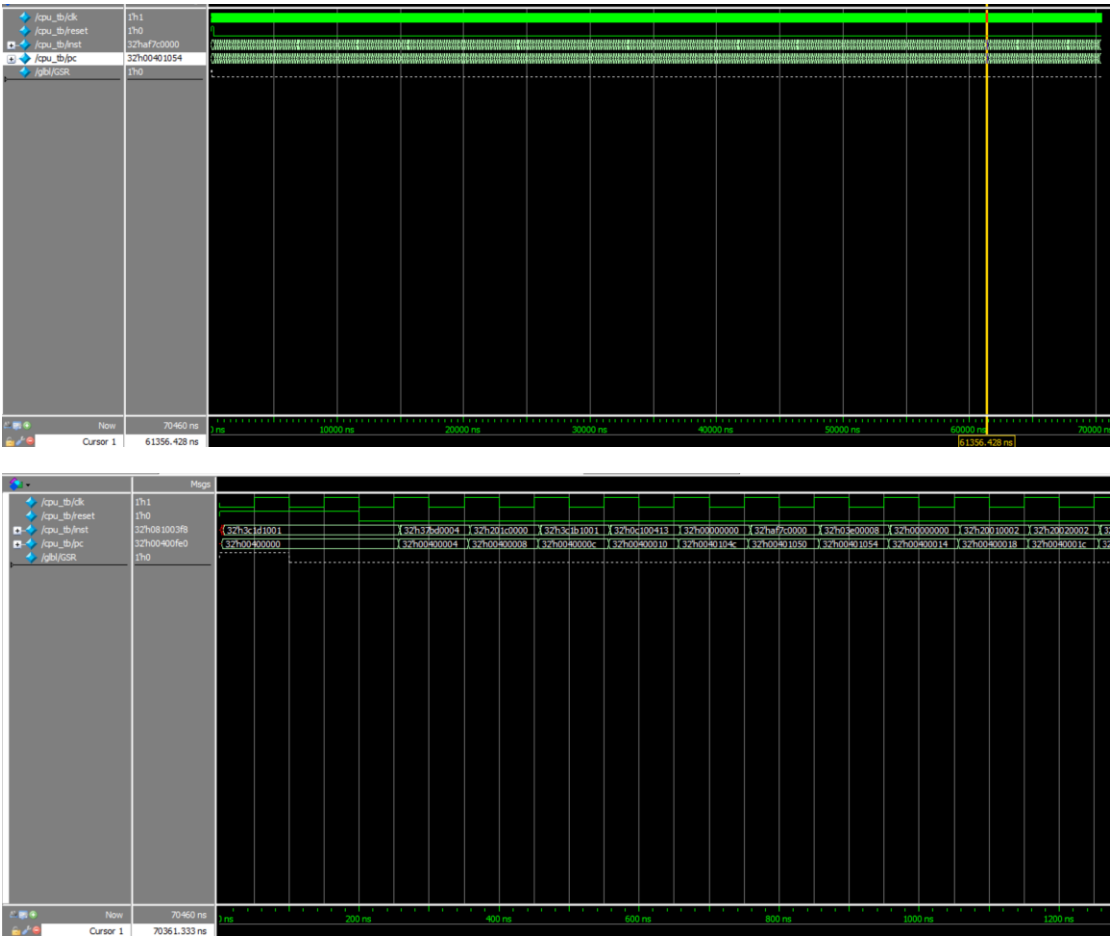
五、CPU 后仿真测试结果

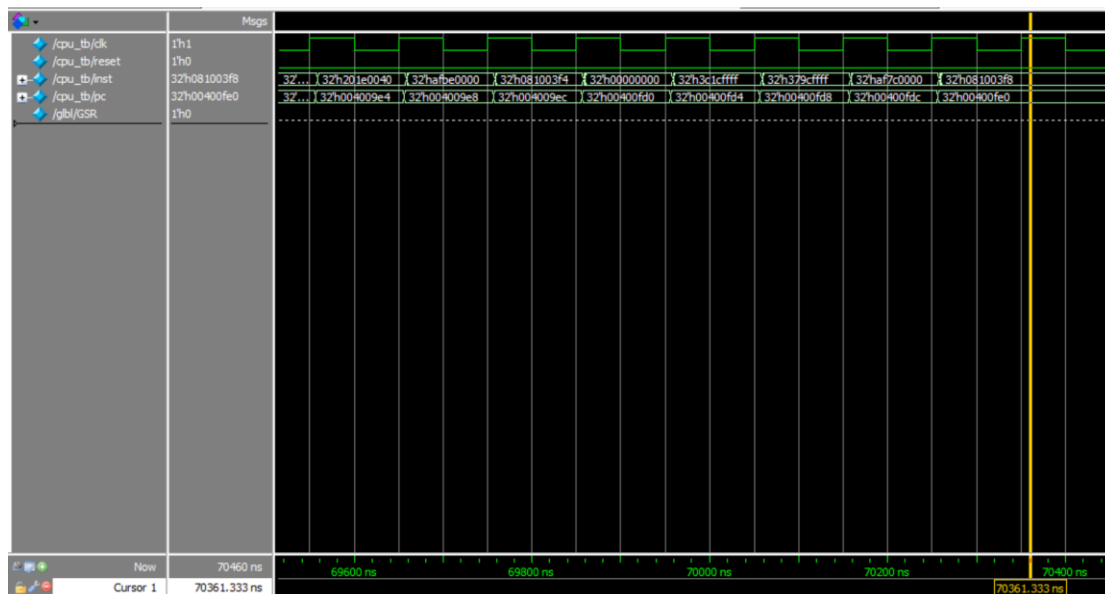
报告要求:

- (1) 对 CPU 后仿真的实验内容和结果进行描述, 并给出 modelsim 后仿真波形图的贴图。
- (2) 对时序分析的结果进行分析, 并截图。

本次 CPU 后仿真在 Vivado 综合后进行。根据实验要求, 后仿真阶段使用由 mips_31_mars_simulate.coe 初始化的 IMEM IP 核, 不再使用 \$readmemh 方式初始化指令存储器。仿真时通过 testbench 提供 clk 和 reset 信号, 运行 Post-Synthesis Timing Simulation, 并通过 ModelSim 波形观察 pc 和 inst 的变化。

后仿真波形图:





从后仿真波形可见，复位释放后 CPU 能够正常取指执行，pc 按程序流程变化。最终 pc 运行到 0x00400fe0，对应指令为 0x081003f8，即程序末尾的自循环跳转指令，说明 CPU 已运行到测试程序结束位置。后仿真结果与前仿真观察结果一致，表明综合后的电路在当前时钟周期下能够正确运行。

时序分析：

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	91.967 ns	Worst Hold Slack (WHS):	0.113 ns	Worst Pulse Width Slack (WPWS):	49.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	137	Total Number of Endpoints:	137	Total Number of Endpoints:	82

All user specified timing constraints are met.

本次时序分析在综合后的顶层文件设计上进行，使用 Vivado 的 report_timing_summary -delay_type min_max 生成时序报告。本设计设置的主时钟 clk_pin 周期为 100ns，对应工作频率 10MHz（直接使用了下发的 idf.xdc）；报告中 WNS = 91.967ns、TNS = 0.000ns，说明建立时间满足要求且不存在 setup 违例；WHS = 0.113ns、THS = 0.000ns，说明保持时间满足要求且不存在 hold 违例；WPWS = 49.500ns、TPWS = 0.000ns，说明脉宽检查也满足要求。最终报告显示“All user specified timing constraints are met”，因此在当前时钟周期约束下，设计满足时序要求。

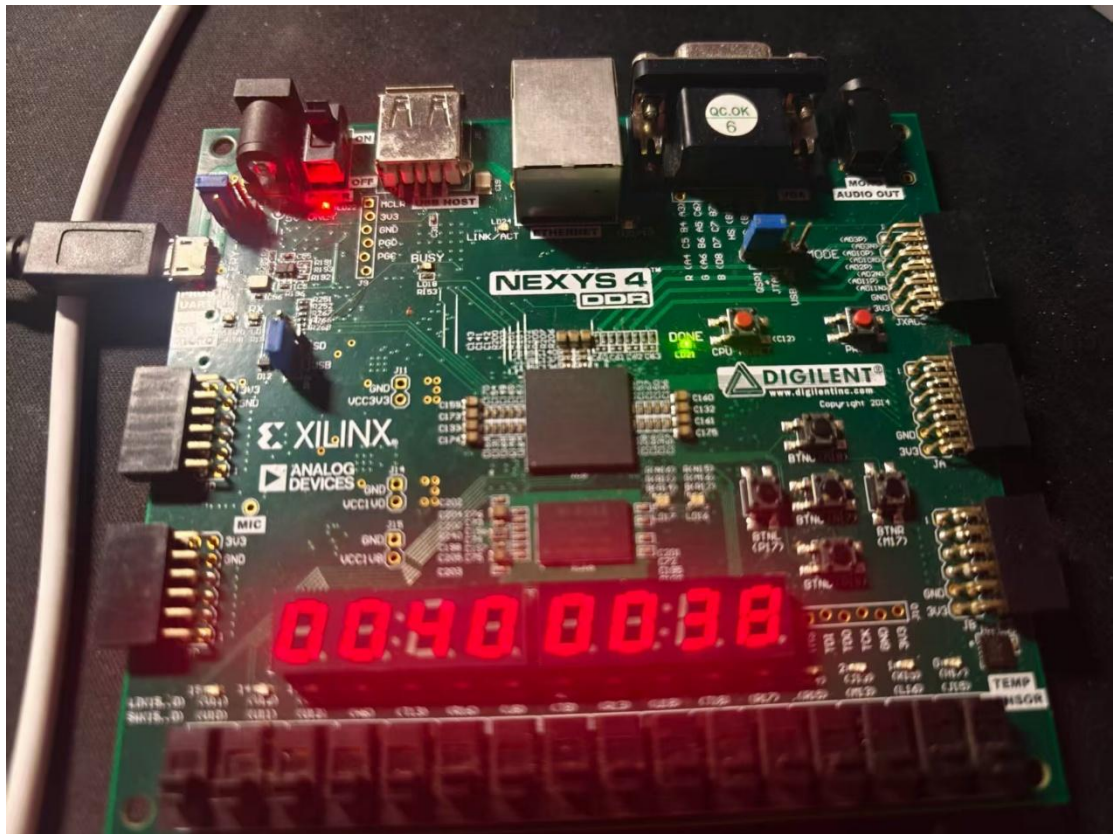
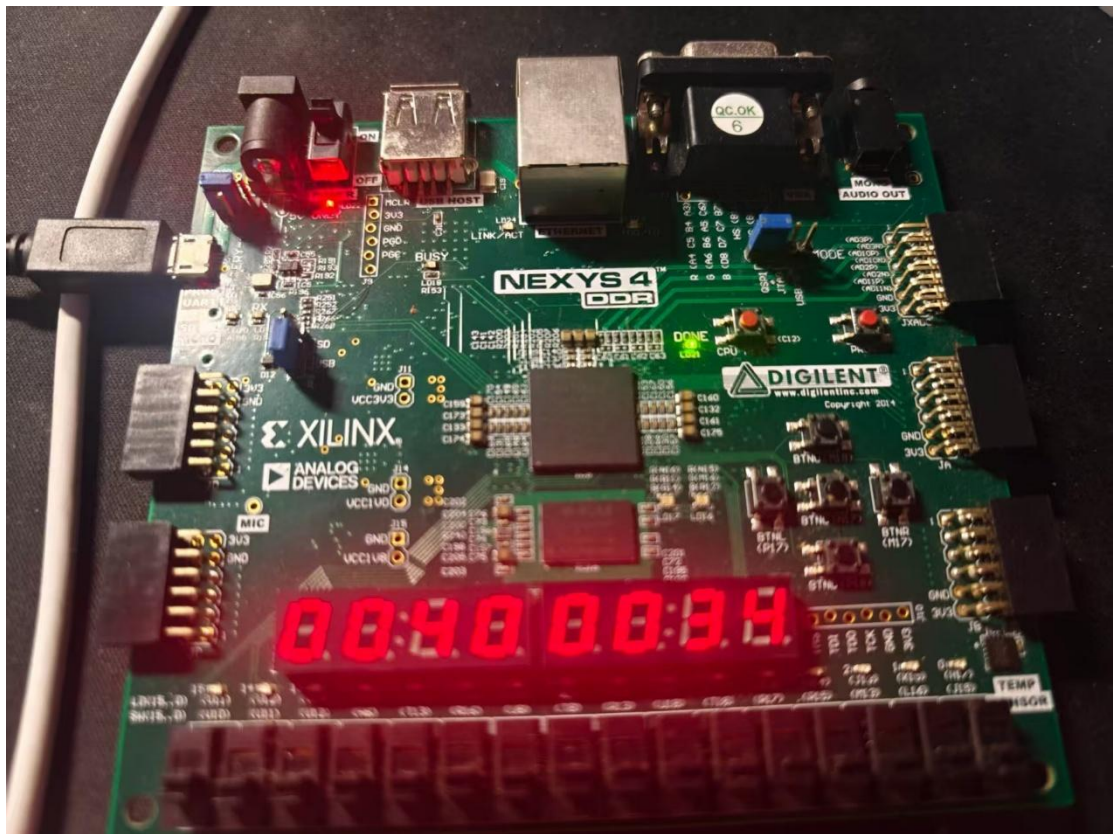
六、CPU 下板结果

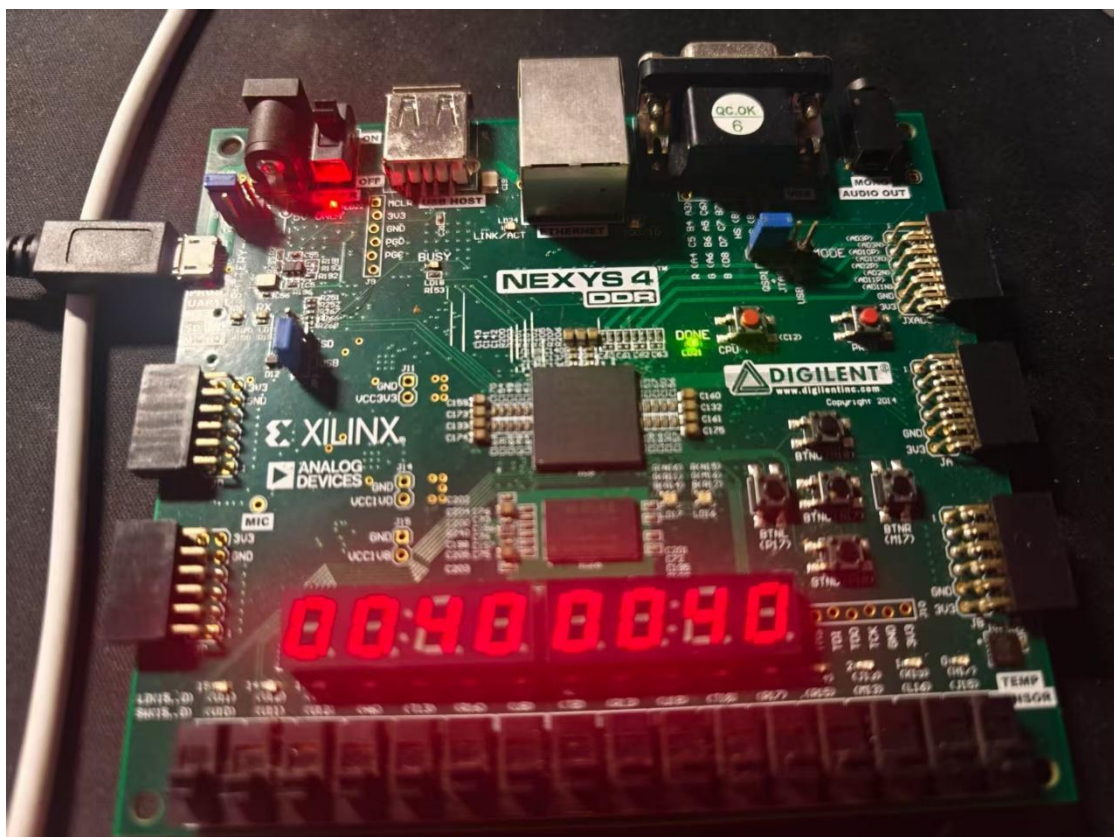
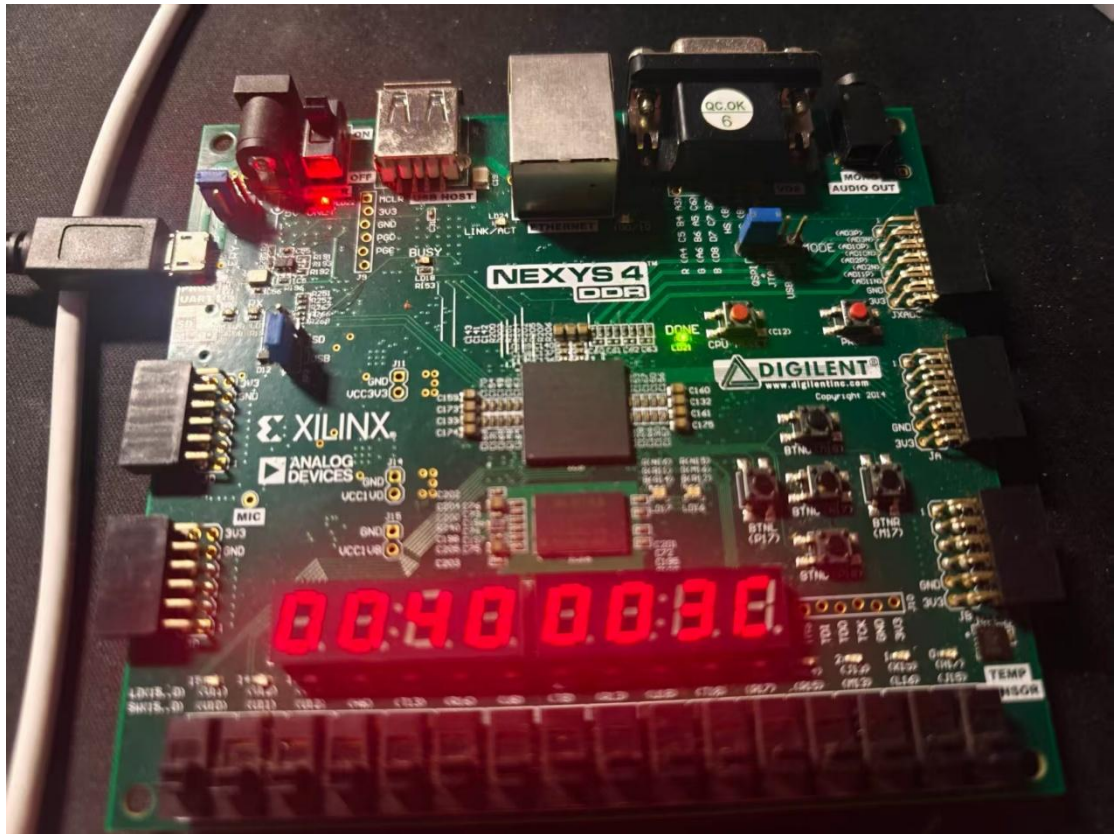
报告要求：

- （1）对 CPU 下板的实验内容和结果进行描述，并给出下板结果的贴图。
- （2）实验的分析与结论。

下板测试时，使用课程提供的 mips_31_mars_board_switch.coe 文件初始化 IMEM 的 IP 核，并通过七段数码管观察 PC 的变化。实际现象为 PC 会在 0x00400034、0x00400038、

0x0040003c、0x00400040 之内死循环。





随后使用相同的 coe 进行后仿真验证，发现程序在 $PC=0x00400040$ 后会出现异常。

	/cpu_tb/reset	1'h0
	/cpu_tb/inst	32'h00000000
	/cpu_tb/pc	32'h00000000
	/gbl/GSR	1'h0

因此该现象推测与下板测试 coe 文件本身或其初始化内容有关，不是 CPU 本身的错误。

reset 能正常工作：

